

THESYS+ — Explained So a Child Can Understand It

This document walks through the THESYS+ codebase one file at a time, in plain language. Think of the whole app as a **restaurant**:

- The **frontend** (in `frontend/`) is the dining room — what you see and touch.
- The **backend** (in `backend/`) is the kitchen — where the real work happens.
- The **code** is the set of written instructions the restaurant runs on.

Each file below is explained line by line. The goal is understanding, not jargon.

How to read the explanations

- A file is introduced with `## path/to/file`.
 - Inside, we go through the file a few lines at a time, like a story.
 - Code words are shown in backticks.
 - Big ideas get a little emoji so they are easy to spot.
-

Part 1 — Frontend foundation

frontend/src/main.jsx — "Turn on the lights" ■

This is where the website starts when you open it.

- **Line 1:** `import { createRoot } from 'react-dom/client';` — Get the tool that plants a React app onto a real web page.
- **Line 2:** `import './index.css';` — Also bring the stylesheet — the book of colors, fonts, and spacing rules.
- **Lines 3–6:** `import App ... ThemeProvider ... AuthProvider ... ToastProvider` — Bring in the main screen (App) and three helper "wrappers" that remember things: Theme (light/dark mode), Auth (are you logged in?), Toast (little pop-up messages).
- **Lines 8–16 (the NOTE):** A warning to future programmers: *We turned OFF React's "StrictMode" on purpose.* Why? In testing mode React runs some code twice. That double-run made the login system think a bad guy was replaying a stolen key, so it kicked everyone out on every page load. So they removed it. (A real bug they fixed!)
- **Lines 17–25:** Take the spot on the page called `root` and draw this ****nest of boxes**** inside it: Theme on the outside, then Auth, then Toast, and right in the middle, the App (all the actual pages). Think of it like: Winter coat (Theme) → sweater (Auth) → shirt (Toast) → you (App) in the middle. Everything inside gets to use what the outside provides.

frontend/src/App.jsx — "The map of the building" ■■

This file decides **which page shows when you visit a web address**.

- **Lines 1–14 (comment):** Explains the plan: the home page / stands alone, all the

sign-in-style pages share one frame, and a single "bouncer" guards the private pages.

- **Lines 16–37:** A big list of `import` lines — "Bring every page and every helper we might need" (`LandingPage`, `SignInPage`, `RepositoryPage`, etc.).
- **Lines 42–65** `NotFound`: If someone types a garbage web address, show a friendly 404 page with the logo and a "Back to Home" button. Like a "Wrong door!" sign.
- **Lines 70–89** `ProtectedRoute`: A bouncer at the door ■. It checks: *Are we still loading?* (show a spinning wheel). *Are you logged in?* If not, send you to the sign-in page. If yes, let you through (`<Outlet />` = "go ahead inside").
- **Lines 91–130** `App`: Here's the actual map:
- **Line 93** `<BrowserRouter>`: Turn on the "address bar understands pages" machine.
- **Line 98**: `/` → `LandingPage` (the welcome home page).
- **Lines 101–107**: `/sign-in`, `/request-access`, `/forgot-password`, `/reset-password`, `/verify-email` → all share one common frame (`AuthLayout` = same header/background). These are the "before you log in" doors. - **Lines 110–118**: `/repository`, `/title-similarity`, `/trend-analysis`, `/analytics`, `/profile`, `/settings` → all behind the **bouncer** (`ProtectedRoute`). You must be logged in. - **Line 121**: Any unknown address → the 404 `NotFound` page. - **Line 125**: One always-available "Upload Thesis" pop-up window, reachable from anywhere.

frontend/src/api/client.js — "The waiter who talks to the kitchen" ■■■

The frontend can't do secret work (like checking a password) itself — it asks the backend. This file is the **telephone** between them.

- **Line 17**: `import axios from 'axios';` — Get the messenger library that sends requests to a server.
- **Lines 19–25**: Create one trusted messenger (`client`) with rules: `baseUrl` (the kitchen's address), `withCredentials: true` ("always bring the secret cookie so the kitchen knows who you are"), `Content-Type: application/json` ("speak in JSON", the universal data language).
- **Line 28**: `let refreshing = null;` — A note saying "are we currently asking for a new key?" (starts as "no").
- **Lines 32–34**: Three empty note-spots: "how to get the current key," "what to do when we get a new one," "what to do if it fails." `AuthContext` will fill these in later.
- **Lines 40–44** `configureAuthCallbacks`: A setup function: `AuthContext` tells the messenger "here's how to grab the key, here's what to do on success/failure."
- **Lines 49–60 (Request interceptor)**: Before every message is sent, automatically staple your ID badge (`Bearer <token>`) onto it — *if* you have one. Like showing your membership card before ordering.
- **Lines 65–72 (Response interceptor, success)**: When a message comes back fine, ring a little bell (`axios: request: success`) so the app knows "user is active, don't log them out from boredom."
- **Lines 73–124 (Response interceptor, error)**: If the kitchen says `**401` = "your key

expired": mark this request "already retried" (so we don't loop forever), single-flight** (if a refresh is already happening, wait for it instead of asking twice), ask the kitchen for a fresh key (/auth/refresh/), staple the *new* key on and re-send your original order. If even the refresh fails: the helpers onRefreshFailure() clear your login, and we send you back to the sign-in page. Finally, erase the "refreshing" note so the next person can try.

- **Line 127:** export default client; — Hand this finished messenger to the rest of the app to use.

Part 2 — The login brain (frontend)

frontend/src/context/AuthContext.jsx — "The brain that remembers who you are" ■

Imagine a name-tag clip on the wall. Everyone in the restaurant checks it to know if you're a member.

- **Lines 1–9 (comment):** This is the *real* login memory. The secret key lives only in the computer's short-term memory (never written on a sticky note/localStorage). The "stay-logged-in" key is a special cookie the browser hides from JavaScript.
- **Line 11:** import { createContext, useContext, useState, useEffect, useCallback } from 'react';
— Get five React tools: a shared memory box (createContext), a way to read it (useContext), sticky notes that change (useState), "do this after something" (useEffect), and "remember this recipe" (useCallback).
- **Line 12:** import client, { configureAuthCallbacks } from '../api/client'; — Bring the waiter (the client from the previous file).
- **Line 13:** import { clearAllCaches } from '../utils/appCaches'; — Bring a "erase the chalkboard" tool so a new person doesn't see the old person's notes.
- **Line 15:** const AuthContext = createContext(undefined); — Make an empty shared box called AuthContext. (undefined) = "nothing inside yet."
- **Line 17:** export function AuthProvider({ children }) { — Here's the Provider — the thing that wraps the app (we saw it in main.jsx, the sweater layer). children = everything inside it.
- **Lines 18–20:** Three sticky notes: user = who you are (starts empty), accessToken = your current entry key (starts empty), isInitializing = "are we still figuring out if you're logged in?" (starts true = yes, still checking).
- **Line 22:** const isAuthenticated = !!accessToken && !!user; — You count as "logged in" only if you have BOTH a key AND a known user. !! just turns it into a yes/no answer.
- **Lines 28–43 loadMe:** Go ask the kitchen "who am I?" ■. It calls GET /auth/me/.
If a token is given, staple it on. The kitchen replies with your profile; we save it in user. If it fails, we forget you (set both to empty) and admit the failure.
- **Lines 48–59 (useEffect):** As soon as this provider turns on, tell the waiter (the client): "Here's how to grab the current key, and here's what to do when you get a new one or fail." This connects the waiter to this brain.
- **Lines 64–87 (silent refresh on startup):** When the app first opens, *quietly* try to

log you back in using the hidden cookie: call `POST /auth/refresh/`, save the new key, load your profile with that key, if anything fails (bad cookie, kitchen on fire, no internet) → forget you, **finally** flip `isInitializing` to `false` so the bouncer stops showing the spinner. (The comment explains a past bug: they used to only forget you on certain errors, which wrongly kicked people out when the wifi blipped. Now they forget you on **any** failure — safer.)

- **Lines 92–109** `signIn`: When you type email + password + maybe "remember me": send them to `POST /auth/login/`, grab the key from the reply, save it, load your profile, if it fails, forget everything and pass the error up so the page can show "wrong password."
- **Lines 114–127** `signOut`: Tell the kitchen `POST /auth/logout/` (best effort — even if that fails we still log you out locally). **Finally** erase all cached pages (so the next person can't peek at your data) and clear key + user.
- **Lines 132–143** `refresh`: Manually ask for a fresh key. Same idea as silent refresh but called on purpose.
- **Lines 145–154**: Pack everything into one `value` box: the user, the key, the yes/no flags, and all four actions (`signIn`, `signOut`, `refresh`, `loadMe`).
- **Lines 156–160**: Hand that box to every children inside the provider.
- **Lines 163–169** `useAuthContext`: A safe way to **open** the box. If someone tries to use it outside the provider (no sweater on), shout an error: "You forgot to wrap me in `AuthProvider!`"

frontend/src/hooks/useAuth.js — "A nickname" ■■

- **Line 10**: `export { useAuthContext as useAuth } from '../context/AuthContext';` — Let people type `useAuth` instead of the long `useAuthContext`. It's just an alias — same thing, shorter name. (Like calling refrigerator just fridge.)

Part 3 — The login kitchen (backend)

backend/auth_service/serializers.py — "The form checker at the door" ■

- **Lines 1–7 (comment)**: Before the kitchen does any real work, this file just checks the **shape** of what you sent. It makes sure your email looks like a school email. It does NOT check your password yet — that happens later, on purpose, to avoid a sneaky trick.
- **Line 9**: `from __future__ import annotations` — A small setting so the file can use modern type hints.
- **Line 11**: `from rest_framework import serializers` — Get Django REST's form-checking tools.
- **Lines 13–16**: `from common.validators import (...)` — Get the rule: "is this a school email?" and the error name for a wrong-domain email.
- **Lines 19–24** `LoginSerializer`: Describe the login form: `email` (must look like an email, max 254 letters), `password` (text, hidden when echoed back, can't be blank, max 512), `remember_me` (a yes/no, optional, defaults to no).

- **Lines 26–33** `validate_email`: When checking the email: trim spaces and make it lowercase, ask "is this a school email (ends with `pampangastateu.edu.ph`)?" If not → shout `INVALID_EMAIL_DOMAIN`, otherwise accept the cleaned version.

Why check email here but password later? The comment explains: to avoid **enumeration leak** — a trick where a hacker tells if an email exists by the **type** of error. By checking the password in the view the same way every time, you can't tell which part was wrong.

backend/auth_service/views.py — "The login counter in the kitchen" ■■■■

- **Lines 1–7 (comment)**: These are the actual login/refresh/logout/me actions. The hard token work is delegated to helper files. Every cookie-using action also checks the request came from our own website (anti-fake-origin guard).
- **Lines 9–35**: A pile of `import` lines — Collect all the tools we'll use: settings, time, response helpers, the User model, password checkers, the audit logger (a security diary), the origin guard, error types, rate limiters, and cookie helpers.
- **Lines 42–54** `_reject_unknown_fields`: If someone sends a field we didn't ask for (like sneaking in `is_admin: true`), reject it with `"UNKNOWN_FIELD"`. Keeps the form strict.
- **Lines 57–64** `_user_payload`: Build the small "here's who you are" card: id, email, first/last name, role.
- **Lines 71–73 (decorators on LoginView)**: Three guards taped above the login door:
 1. `require_origin_match` — only accept requests from our own site,
 2. `rate_limit_per_ip(10, 60, ...)` — one IP can try only **10 times per 60 seconds**,
 3. `rate_limit_per_email_on_failure(5, 15*60, ...)` — after **5 wrong tries on an email**, lock it for **15 minutes**. (These stop hackers from guessing passwords thousands of times a second.)
- **Lines 74–83** `LoginView`: The login counter. `AllowAny` = anyone may **try**. No built-in auth needed yet.
- **Lines 85–88**: First, reject unknown extra fields.
- **Lines 90–106**: Check the form is valid (email format, etc.). If the email field is bad, return a tidy error; otherwise a generic "invalid" error. We keep errors vague on purpose.
- **Lines 108–110**: Pull out the cleaned email, password, and `remember_me`.
- **Lines 112–115**: Look up the user by email. If found and active, check the password (`accounts_verify_password`). `needs_rehash = "should we upgrade how we stored this password?"`
- **Lines 117–133**: If the password is wrong: write in the security diary `auth.login.failure` (note the email tried and the IP — for safety investigators), then reply "Email or password is incorrect." with code `INVALID_CREDENTIALS`. Same message whether the email didn't exist or the password was wrong — no hints!
- **Lines 135–136**: If the password-checker says "please re-store this password more safely" (`needs_rehash`), do it quietly.
- **Lines 138–139**: Stamp `last_login_at` = right now, and save just those two fields.
- **Line 141**: Ask the token service to **issue a token pair** (a short key + a long hidden cookie key).

- **Lines 143–150:** Build the reply: the access key, "Bearer" type, how many seconds it lasts, and your user card. Then **bake the refresh key into a hidden cookie** on the reply (`set_refresh_cookie`).
- **Lines 152–159:** Write `auth.login.success` in the diary (with your id and the "family id" of this session). Then send the reply.
- **Lines 167–173** `RefreshView`: The "give me a new short key" counter. Same origin guard; allows **60 refreshes per 60 seconds per IP**.
- **Lines 175–182:** Read the hidden refresh cookie. If it's missing → say "REVOKED / cookie missing".
- **Lines 184–206:** Try to rotate the refresh key. Three possible problems: **Expired** → "your long key timed out," **Reuse detected** → "someone tried to use an old key twice — we threw away the WHOLE session family" (this is the security feature that forced `main.jsx` to drop `StrictMode!`), **Revoked** → "this key was cancelled." Each writes a diary entry.
- **Lines 208–215:** If all good: reply with a new access key and bake a fresh refresh cookie.
- **Lines 222–240** `LogoutView`: Logout: read the cookie, if present cancel that one key row (`revoke_one`), reply "204 no content", and **wipe the cookie** (`clear_refresh_cookie`). If a key was actually cancelled, note `auth.logout` in the diary. Friendly: even if the cookie was already gone, logout still succeeds.
- **Lines 247–258** `MeView`: The "who am I?" counter. `IsAuthenticated` = you must already be logged in. It replies with your user card **plus** your list of permissions (what you're allowed to do, based on your role).

Part 4 — Inside the login kitchen (backend `auth_service` internals)

backend/auth_service/models.py — "The key notebook" ■

This is the **notebook** where the kitchen records every "stay-logged-in" key. The most important safety rule: only a **scrambled** version of the key is written down — never the key itself.

- **Lines 1–11 (comment):** The notebook stores "opaque refresh tokens." Each key has a "family." When you log in you get one key; when it's replaced (rotated) the new key joins the same family. If someone re-uses an **old** key, the whole family is cancelled.
- **Lines 13–18:** Imports — `uuid` (random IDs), Django settings, and the model tools.
- **Lines 21–28** `class RefreshToken`: The table. `REVOKED_REASON_CHOICES` is the list of reasons a key was cancelled: `rotated`, `logout`, `reuse_detected`, `password_reset`, `admin_revoke`.
- **Line 30:** `id` — every row gets a random UUID name tag.
- **Lines 31–36:** `user` — which person this key belongs to. If that person is deleted, their keys are deleted too (CASCADE).
- **Line 37:** `token_hash` — the **scrambled fingerprint** of the key (64 letters,

unique). The real key is never saved — only this fingerprint.

- **Line 38:** `family_id` — the "family group" ID. All keys from one login share it.
- **Lines 39–46:** `parent` — points at the *previous* key, like a family tree. When we replace a key, the new one's parent is the old one.
- **Line 47:** `remember_me` — did the user tick "keep me logged in?"
- **Line 48:** `expires_at` — the date/time the key stops working.
- **Lines 49–55:** `revoked_at` (when it was cancelled, empty if still good) and `revoked_reason` (one of the listed reasons, or empty).
- **Lines 56–58:** `created_at`, `user_agent` (which browser), `ip_address` (which computer) — written down for the security diary.
- **Lines 60–88** `class Meta`: Extra rules for the table: its real name is `refresh_tokens`; it builds **indexes** (fast lookup lists) for "this user's live keys" and "this family"; and a **check constraint** that `revoked_reason` must be one of the five reasons or empty (no typos allowed in the diary).
- **Lines 90–91:** `__str__` — how to print a row as text (used in logs).

backend/auth_service/services.py — "The token factory" ■

This file is the machine that **makes and cancels** keys.

- **Lines 1–11 (comment):** Lists the five jobs this factory does: issue a pair (at login), rotate (at refresh), revoke a whole family, revoke everything for a user, revoke just one key (logout).
- **Lines 13–31:** Imports and grabbing helper tools (the JWT maker, the opaque-token maker, the scrambler sha256).
- **Lines 34–38:** The five cancel-reason labels, written as constants so we don't mistype them.
- **Lines 41–47** `IssuedTokenPair`: A small labeled box that holds: the access key, the plain refresh key (the real one, only given to the browser), and the notebook row we created.
- **Lines 50–55** `_ttl_for`: "How long should the refresh key live?" Longer if `remember_me` is on.
- **Lines 58–71** `_client_ip / _user_agent`: Figure out *which computer* and *which browser* is asking, for the diary. (The X-Forwarded-For trick just reads the first IP if the request came through a proxy.)
- **Lines 74–90** `issue_token_pair`: At login: make an access JWT, make a random refresh key, store only its **fingerprint** in a fresh notebook row (new family ID, no parent), and hand back the pair.
- **Lines 93–97** `revoke_family`: Cancel **every still-good key** in a family in one database update. (Used when we suspect a stolen key.)
- **Lines 100–104** `revoke_all_for_user`: Cancel every key for one person (used by password reset — when you change your password, all old sessions die).

- **Lines 107–175** `rotate_refresh`: The clever one. It exchanges an old key for a new pair, and watches for theft: - **Line 121**: Scramble the presented key into a fingerprint. - **Line 126**: Open a **locked drawer** (`transaction.atomic`) so two requests can't trip over each other. - **Lines 128–130**: Find the notebook row by fingerprint; if not found → "Revoked." - **Line 132–133**: If expired → "Expired." - **Lines 135–144**: If already cancelled: **only** if the reason was rotated do we treat it as **theft (reuse)** — remember the family to kill it; otherwise just "Revoked." - **Lines 145–169**: If active → **rotate**: mark the old row cancelled (reason rotated), make a brand-new key row that **inherits the same family_id and remember_me**, issue a new access key, and return the pair. - **Lines 174–175**: After leaving the locked drawer, if we detected reuse, cancel the **whole family** and shout `RefreshReuseDetected`. (This is the rule that forced `main.jsx` to drop `StrictMode` — a test double-run looked like theft and booted everyone!)
- **Lines 178–192** `revoke_one`: Logout — cancel just this one row. Friendly: it does **not** error if the key is missing/expired/already cancelled.

backend/auth_service/authentication.py — "The badge checker" ■

- **Lines 1–14 (comment)**: This reads the `Authorization: Bearer <key>` header and checks the key is real. If the key is **expired**, it raises a special error the frontend recognizes so it can quietly fetch a new one. (Imports are done lazily inside the function to avoid a "circular import" tangle — two files each needing the other.)
- **Lines 18–25**: Imports and `class JWTAuthentication` with `keyword = 'Bearer'`.
- **Lines 27–60** `authenticate`:
- **Line 29–35**: Lazy imports (`User`, the `expired-error`, the `JWT verifier`).
- **Line 37**: Read the `Authorization` header.
- **Line 38–39**: If there's no header → return `None` ("not logged in yet; the bouncer will decide"). - **Lines 41–47**: Split the header; it must say `Bearer`, and have exactly the key (not zero, not three parts) — otherwise complain. - **Line 49–55**: Take the raw key, verify it. If expired → raise the special expired error (frontend refreshes). If invalid → fail. - **Lines 57–59**: Look up the user by the ID hidden in the key; if missing or turned off → fail. - **Line 60**: Return (`user`, `payload`) — "yes, this person is who they claim to be."
- **Lines 62–63** `authenticate_header`: Just returns the word `Bearer` for error messages.

backend/auth_service/urls.py — "The door signs" ■

- **Lines 1–6 (comment)**: This file maps web addresses to the counters. The school-login (SSO) doors live in their own spot so swapping in a real school provider later won't disturb the normal login.
- **Lines 8–11**: Import the `path` tool and the `counters/views` (including the SSO views).
- **Lines 14–21** `urlpatterns`: The list of doors:
- `login/` → `LoginView`
- `logout/` → `LogoutView`
- `refresh/` → `RefreshView`
- `me/` → `MeView`
- `sso/initiate/` and `sso/callback/` → the school-login views

Part 5 — Frontend public pages (before you sign in)

frontend/src/pages/LandingPage.jsx — "The restaurant's front window display" ■

This is what a visitor sees at / before ever signing in — the whole "menu board" for THESYS+.

- **Lines 1–15 (comment):** Explains the layout: sticky navbar, a split hero (text on the left, a photo of the CCS building on the right), a repository snapshot strip, a 4-card feature showcase, trending topics + "why THESYS+", and a dark footer. **Privacy note:** logged-out visitors only ever see **aggregate** counts — never real thesis titles or authors, like a restaurant showing "500 meals served" on a sign without printing anyone's receipt.
- **Lines 33–39 CORE_NAV:** The list of top navigation links (Home, Repository, Title Similarity, Trend Analysis, Analytics) — every one is already built (`implemented: true`), so nothing here is a "coming soon" placeholder.
- **Lines 44–81 FEATURES:** Four feature cards' worth of icon + color + title + description + link — the four things THESYS+ can do, described in one sentence each.
- **Lines 88–96 (state):** Search box text, thesis count, trending topics, which legal popup is open, mobile menu open/closed, whether the building photo loaded. `isDark` just mirrors the current theme.
- **Lines 98–112 (effects):** Three independent "as soon as the page opens, go find out..." jobs: (1) let Escape close the mobile menu, (2) **probe** whether `ccs-bldg.jpg` actually exists (some deployments won't have it — falls back to a plain blue tint instead of a broken image icon), (3) nothing else here yet.
- **Lines 115–139 (public data fetches):** Ask the kitchen for two numbers
without needing to be logged in: `GET /theses/public-stats/` (how many theses are indexed) and, only if you **are** signed in, `GET /theses/topic-trends/` (a small trending-topics preview). Both fail silently (`.catch(() => {})`) — a slow API service should never break the homepage.
- **Lines 143–166 (search + trend helpers):** Typing in the hero search box and hitting Enter routes you to `/repository?q=<your text>`. `trendChip` picks a color/label for each trend classification; `TREND_FALLBACK` is a hard-coded set of plausible topics shown before real data has ever loaded, so the page never looks broken on a fresh install.
- **Lines 168–423 (Navbar + Hero):** The sticky top bar (mobile hamburger + drawer on small screens, full link row on desktop), then the two-column hero: headline, one-line pitch, the search bar, and a "Check Title Similarity" button. The CCS building photo sits on the right with a soft left-edge fade so text and photo blend into one image, like a photo faded into a painted wall.
- **Lines 428–493 (Repository Snapshot):** Three stat tiles: total indexed theses, the year range covered, and the three programs the corpus spans.
- **Lines 498–521 (Feature Showcase):** Renders the four FEATURES cards.
- **Lines 526–615 (Trending Topics + Why THESYS+):** A live/-ish preview of trending topics next to three short "why we built this" pitches (prevent duplicate topics, improve discoverability, reveal trends).
- **Lines 620–699 (Footer):** Institution name, quick links that open the

LegalModal (About/Privacy/Terms/Help — pop-ups, not separate pages), real outbound links to the university site and CCS Facebook page, and a copyright line.

- **Line 702** <LegalModal>: One shared popup component reused for all four footer links — only its type prop changes which text it shows.

frontend/src/pages/ForgotPasswordPage.jsx — "I lost my key, kitchen — help!" ■

- **Line 18** INSTITUTIONAL_EMAIL_RE: A pattern double-checking, right in the browser, that the typed email ends in pampangastateu.edu.ph — instant feedback before ever asking the kitchen.
- **Lines 20–26** mapError: Turns a raw error code into a friendly sentence (wrong domain / too many tries / anything else).
- **Lines 32–36 (state)**: The typed email, its validation message, loading flag, a generic error, and success (did the kitchen say "okay, sent")?
- **Lines 47–60** handleSubmit: Validate the email shape, then POST /auth/forgot-password/. **On success we don't reveal whether the email was real** — the confirmation screen (lines 77–97) always says the same thing ("if an account exists...") so an attacker can't use this form to find out who has an account. This mirrors the backend's own "always 200" behaviour (see ForgotPasswordView further down).
- **Lines 98–141 (the form)**: One email field, a small "links expire after 30 minutes" trust cue, and a submit button — nothing else to see until you submit.

frontend/src/pages/RequestAccessPage.jsx — "Filling out the membership form" ■

This is the very first step of onboarding — before a person has any account at all.

- **Lines 21–35** mapError: A dictionary translating ~8 backend error codes (wrong email pattern, duplicate request, file too large, unsupported file type, etc.) into plain sentences.
- **Lines 37–62** DECISIONS: Three possible outcomes after you submit your ID document, each with its own icon, color, headline, and explanation: - pending_email_verification — "your document looked genuine automatically, check your email" (green, success tone). - pending_manual_review — "a human needs to look at this" (amber, warning tone). - rejected — "we couldn't confirm you're PampangaStateU/CCS" (red, error tone).
- **Lines 86–165** ProgressStepper: The reusable 4-node "Submit → Verify Email → Set Password → Account Ready" visual you'll also see on VerifyEmailPage. When the outcome is "manual review," step 2's label temporarily swaps to "Manual Review" instead of "Verify Email" so the wording matches what's actually happening.
- **Lines 178–191** handleSubmit: Sends the whole form — including the uploaded ID photo/PDF — as multipart/form-data to POST /auth/request-access/, then remembers which decision came back.
- **Lines 211–306 (render)**: Shows either the form (RequestAccessForm, a separate component covered in Part 7) or, once a decision exists, a colored result card with the matching icon/title/body/note from DECISIONS.

■ ■ **Audit finding (fixed during this audit):** line 247 used to say *"After clicking the verification link, you will receive a second email to set your password."* — leftover copy from before the single-email fix (see Part 5's `VerifyEmailPage` below), where clicking the link now lets you set your password **on the same page** and no second email is ever sent. Corrected to: *"After clicking the verification link, you'll set your password right on that same page — no second email needed."*

frontend/src/pages/ResetPasswordPage.jsx — "Forgot-password key, act two" ■

Landed on from the **forgot-password recovery email** (not the single onboarding email — that one goes to `SetupAccountPage` instead).

- **Line 18** `validateStrength`: The same "12+ characters, a letter, a digit" rule enforced everywhere passwords are set.
- **Lines 62–65 (invalid token handling)**: If there's no `?token=` in the URL at all, immediately show the "link invalid or expired" state — no point showing a form that can't work.
- **Lines 67–84** `handleSubmit`: Validate strength and match locally first (no wasted trip to the kitchen), then `POST /auth/reset-password/` with the token + new password. Success sends you to `/sign-in?reason=password_set` (a green banner on the sign-in page). A backend `INVALID_RESET_TOKEN` response flips the whole card into the "expired" state with a link back to `/forgot-password` to request a fresh one.
- **Lines 99–202 (render)**: A standard two-field password form with show/hide eye icons, inline strength hint, and a "Reset Password" button — or, if the token's bad, a dead-end card pointing back to `Forgot Password`.

frontend/src/pages/SetupAccountPage.jsx — "Open the crate, set up your spot" ■

This is the page the **single onboarding email** ("Welcome to THESYS+ - Set Up Your Account") actually links to — `/setup-account?token=...` It's a near twin of `ResetPasswordPage` but with wording aimed at brand-new accounts instead of password recovery, and it posts to a different endpoint.

- Same `validateStrength` / show-hide-password machinery as `ResetPasswordPage`.
- `handleSubmit`: `POST /auth/setup-password/` (not `/auth/reset-password/`) with `{ token, new_password }`. On success, redirects to `/sign-in?reason=account_setup`, which shows *"Account activated successfully! Please sign in with your new password."*
- **Invalid/expired token state**: Explains that the account already exists (because approval already created it) — so the recovery path here is "Forgot your password?" rather than "submit a new access request," since a brand-new request isn't what's needed; the account is real, it just never got a password set in time.
- **Why a near-duplicate of `ResetPasswordPage` instead of reusing it?** So the wording, heading ("Set Up Your Password" vs "Reset your password"), and success message can each say exactly the right thing for **why** the visitor is here, without an `if/else` branching a shared component in confusing ways.

frontend/src/pages/VerifyEmailPage.jsx — "Click the link, finish signing up right here" ■

This page used to just say "check your email for a second message" — it now does the **whole rest of onboarding inline**, which is the heart of the single-email fix.

- **Lines 1–13 (comment)**: Explicitly documents the new contract: the token in

the emailed link both proves you own the address **and** (if valid) unlocks a password-setup form on this exact page. The backend hands back a `setup_token` in the same response that confirms the email — so a second email is never dispatched.

- **ProgressStepper**: Same reusable 4-step component as `RequestAccessPage`.
- **State machine** — state: `loading` → `set_password` → `success`, with error reachable from `loading` if the link itself was bad. - `loading`: call `GET /auth/verify-email/?token=...` On success, stash `res.data.setup_token` and move to `set_password`. On failure, map the error code (`TOKEN_EXPIRED` / `TOKEN_ALREADY_USED` / `TOKEN_INVALID`) to a plain sentence and show the error state (with "Submit a New Request" — the email-verification token itself is unrecoverable, unlike a setup-password token). - `set_password`: Renders the New Password / Confirm Password form directly in the card — no navigation, no second email. Submitting posts to `POST /auth/setup-password/` with `{ token: setupToken, new_password }`. If *that* fails with `INVALID_RESET_TOKEN` (the 30-minute setup window expired while the form was open), it shows a **local** "Link expired" card pointing at `Forgot Password` — it does **not** fall back to the generic error state, because the email itself is still verified; only the follow-on token timed out. - `success`: The Step-4 "Account Ready" screen with a checkmark and a "Go to Sign In" button that navigates to `/sign-in?reason=account_setup`.
- **activeStep (near the bottom)**: 2 while the password form is showing, 3 once it succeeds — driving which stepper node lights up.

Part 6 — Frontend protected pages (the actual app, behind the bouncer)

frontend/src/pages/RepositoryPage.jsx — "Browse the whole cookbook shelf" ■

The main thesis browser: search, filters, a similarity-threshold slider, and pagination.

- **Lines 1–16 (comment)**: Explains three performance choices: the similarity slider is **debounced** 500 ms (dragging it doesn't spam the server on every pixel), a background refresh keeps old results on screen instead of flashing a blank loading state, and search/filter/page combinations are cached in memory for 2 minutes.
- **Lines 59–73 (module-level cache)**: `repoCache` is a plain JS Map that lives **outside** the component (so it survives re-renders, but is wiped on a full page reload). `getCache/setCache` implement the 2-minute time-to-live. `registerCacheClearer` hooks this cache into the shared "wipe everything on logout" mechanism (see `utils/appCaches.js` in Part 9) so the next person to use this browser never sees a previous user's search results.
- **Lines 79–114 (badge/color helpers)**: Small functions that pick a color for a thesis's status (`approved/pending/rejected`) and for its semantic-match score (green $\geq 80\%$, amber $\geq 60\%$, blue below that).
- **Lines 120–210 ThesisCard**: One clickable card per thesis — title, optional "NN.N% Semantic Match" badge with a tooltip showing the exact cosine score, status badge, program/year, up to 3 authors, up to 4 keyword chips.
- **Lines 225–256 (state)**: Search text (both the **typed** value and the **committed** value — they're different so typing doesn't fire a search on every keystroke), year/program filters, page number, total count, and two threshold values: `sliderThreshold` (what the slider visually

shows **right now**) vs `committedThreshold` (what's actually sent to the API, 500 ms after you stop dragging).

- **Lines 248–256** `handleThresholdChange`: The debounce itself — every time the slider moves, cancel any pending timer and start a fresh 500 ms one.
- **Lines 261–317** `loadTheses`: Build the query string, check the cache first (instant, no spinner), otherwise call `GET /theses/?...` Uses a **soft-loading** flag instead of the full skeleton screen when results are already visible — so background refreshes don't cause a jarring flash. Cancelled requests (component unmounted mid-fetch) are silently ignored.
- **Lines 330–343 (example queries + one-click threshold fix)**:
`runExampleQuery` fills in a known-good search with one click; `applyThreshold` is the "Try lowering the threshold to 50%" button shown on a zero-result search — one click, no manual slider dragging.
- **Lines 370–643 (render)**: Heading + live counts, the filter bar (search + year + program + submit + the similarity slider), cold-start example chips when no search has run yet, then either a skeleton grid, an error banner, one of two different "no results" empty states (zero matches for **this** search vs. an entirely empty repository), or the actual results grid with pagination.

frontend/src/pages/ThesisDetailPage.jsx — "One recipe card, open to read" ■

The single-thesis view. **This is where the watermarked-preview feature lives** — see the recent-changes note below.

- **Lines 1–8 (comment)**: States plainly that the PDF opens in a ***dedicated, watermarked, full-screen tab*** (`PdfPreviewPage`) rather than inline on this page, and that raw downloads are intentionally never exposed.
- `isSaved / toggleSaved`: Read/write the user-scoped "saved theses" list in `localStorage` (see `utils/userStorage.js`, Part 9) — bookmarking a thesis doesn't touch the backend at all in this phase.
- **Two data-loading effects**: one loads the thesis metadata (`GET /theses/{id}/`), the other re-checks whether **this** thesis is in the signed-in user's saved list once user becomes available (it may still be `null` while auth is initializing).
- `handlePreview`: `window.open('/theses/{id}/preview', '_blank', 'noopener,noreferrer')` — opens the dedicated preview page in a brand-new tab. `noopener,noreferrer` stops the new tab from being able to reach back and manipulate the tab that opened it — a standard safety habit for `window.open`.
- **Render**: status/program/year badges, title, authors/adviser, the abstract in a comfortable 65-character reading column (a real typography detail — lines that are too wide are tiring to read), keyword chips, then a footer row with "Uploaded by ... · date," a **Save** toggle button, and the **Preview Document** button (with an Eye icon) sitting right beside it.

frontend/src/pages/PdfPreviewPage.jsx — "The reading room, all to yourself" ■

The dedicated full-screen viewer opened by the button above. Deliberately has **no** navbar or page shell — it's meant to feel like a distraction-free document viewer, not another app screen.

- ***Why fetch the PDF as a *blob* instead of pointing the `<iframe>` straight*

at the API URL? Because the download endpoint requires a login token sent in an Authorization header, and a plain `<iframe src="...">` has no way to attach custom headers. So the page fetches the bytes itself (using the same authenticated client as everywhere else), wraps them in a Blob, and turns that into a temporary blob: URL the `<iframe>` *can* load — like fetching the recipe card yourself and handing the finished page to the reading stand, instead of asking the stand to fetch it (which it can't).

- **The `#toolbar=0&navpanes=0&statusbar=0` suffix:** A hint many PDF viewers understand, hiding the built-in toolbar/side-panel/status-bar so the document fills the space cleanly.
- **The watermark:** Rendered by the shared `WatermarkOverlay` component (Part 7) as an `absolute inset-0` layer sitting *on top of* the `iframe`, not inside it — so it stays fixed over the whole viewport regardless of how far the reader scrolls through the document inside the `iframe` (the overlay isn't scrolling with the content; the outer container is, and the `iframe`'s *own* internal document is what actually scrolls through the pages).
- `handleClose`: Just `window.close()` — works because this page is only ever reached via `window.open` from `ThesisDetailPage`, and browsers only allow a tab to close itself if script opened it.
- **Two loading states:** the thesis title (cosmetic — used only for the header and the `<iframe title>` for accessibility) and the PDF bytes themselves (the one that actually matters — errors here show a friendly "Preview unavailable" message instead of a blank white `iframe`).

frontend/src/pages/TitleSimilarityPage.jsx — "Has anyone cooked this dish before?"

Lets a student check whether their proposed thesis title overlaps too much with existing research, **before** they commit to it.

- **Lines 1–11 (comment):** Two independent ways to get a title into the box: type it, or upload a proposal document and let the backend guess the title for you. Term analysis (shared vs. distinctive keywords) runs **entirely in the browser** — no extra server round-trip.
- `splitTerms` (**lines 34–45**): A small local text-processing function: strip common "stop words" (a, the, of, ...), then compare the proposed title's remaining words against every word appearing in the *matched* theses' titles. Words that also appear in matches are "common terms" (this may be why it's similar); words that don't are "distinctive terms" (this is what makes it different).
- `statusVisuals` (**lines 51–76**): Maps the three risk buckets — Highly Similar (■ ≥ 85%), Moderately Similar (■ 60–84%), Low Similarity (■ < 60%) — to colors and an emoji.
- `handleSubmit` (**manual path**): requires at least 5 characters, then `POST /theses/validate-title/`. The result includes a similarity score, a classification, a recommendation sentence, and a list of matching existing theses.
- `handleFileChange` / `handleExtract` (**upload path**): Client-side checks file type (PDF/DOCX only) and size (≤ 15 MB) before ever uploading, then `POST /theses/extract-title/` with the file. The response includes a **confidence** level (high / medium / low) — low confidence shows a loud warning that the detected text might just be a chapter heading, not the real title, and asks the student to double-check it before validating.
- **Render:** a two-column layout — the form + results on the left,

a "How this works" explainer + (once you validate) a colored Risk Level card on the right — followed by a full-width "Related Existing Studies" grid and the client-computed Term Analysis panel underneath.

frontend/src/pages/TrendAnalysisPage.jsx — "Which recipes are everyone cooking lately?" ■

Shows which research topics are oversaturated, actively growing, or barely explored — an AI-generated map of the whole repository's research landscape.

- **Lines 1–21 (comment):** Explains the 5-minute in-memory cache (same pattern as RepositoryPage's cache, just simpler — one cached blob instead of many keyed queries) and that cached data renders instantly on revisit while a background refresh runs silently.
- **trendStyles:** Maps SATURATED / EMERGING / UNDEREXPLORED to emoji/color/accent, mirroring the backend's own trend classification logic (see theses/services/topic_analysis.py in Part 14).
- **buildDoughnutSegments (lines 105–140):** If there are more than 7 topic clusters, showing all of them as doughnut-chart slices would turn into unreadable confetti. So this keeps the 6 biggest clusters as their own slices and **merges everything else into one gray "Other" slice** — the same trick a pie chart uses when there are too many tiny categories to label individually.
- **DoughnutChart / CountsBarChart:** Both are **hand-drawn inline SVG** — no chart library was pulled in. The doughnut computes each slice's arc length from its share of the total and offsets it around the circle; the bar chart just scales each bar's width to the largest value in the set. Both include an `aria-label` summarizing all the data in plain text, so a screen reader user gets the same information a sighted user gets from the picture.
- **Render:** overview stat cards (total theses, total topics, saturated count, underexplored count), the doughnut + bar chart side by side, a grid of ClusterCards (topic name, trend badge, thesis count, top TF-IDF keywords, up to 3 sample titles), and a "How this works" explainer describing the TF-IDF + K-Means methodology in one paragraph.

frontend/src/pages/AnalyticsDashboardPage.jsx — "The restaurant's monthly report" ■

Deliberately **separate** from Trend Analysis: Trend Analysis is AI **interpretation** (topic clusters); Analytics is raw repository **counting** (totals, distributions, growth over time) — the comment at the top of the file spells this distinction out explicitly so future readers don't merge the two pages by mistake.

- **Module-level cache:** Same 5-minute TTL pattern as TrendAnalysisPage.
- **HBarChart / YearBarChart:** Two more hand-drawn inline SVG charts — a generic horizontal bar (used for both "theses per program" and "top keywords") and a per-year vertical bar chart for growth over time.
- **StatCard:** A reusable number-plus-label tile; some cards (like "Semantic Ready") carry a tooltip explaining exactly what the number means, since "Semantic Ready" isn't self-explanatory to a new user — it means "has an SBERT embedding, so it's searchable by meaning."
- **shortProgram:** Abbreviates long program names ("BS Information System" → "BSIS") purely for compact chart labels.
- **Render:** overview cards (Total Theses, Semantic Ready, Most Active

Program, Recent Uploads, and — if the signed-in role can see it — Pending Review count), the program/growth chart row, a top-keywords bar chart, and a compact TrendSummary (emerging/saturated/underexplored mini-cards) with a link through to the full Trend Analysis page for the deeper view.

frontend/src/pages/ProfilePage.jsx — "Your table at the restaurant" ■

- **Lines 1–16 (comment):** Saved theses live in `**user-scoped localStorage` (keyed by user ID or email) — explicitly flagged as a Phase-1 shortcut; a real production version would store this server-side per account instead.
- **Sidebar:** Avatar (via the shared `useProfilePicture` hook — Part 9), name, role badge, email, bio, an "Edit Profile" link to Settings, and two quick-stat tiles (Saved count, Uploaded count).
- **SavedCard:** One bookmarked thesis — title (links to its detail page), a remove (×) button, program/year/authors, keyword chips, and "Saved <date>".
- **Two tabs — Profile Overview / Saved Theses:** Overview shows account fields (name, email, role, department, research interests, bio — the extras come from `localStorage`, not the backend) plus up to 5 recently uploaded theses fetched live from `GET /theses/?page_size=5`. Saved Theses lists everything bookmarked, with a "Clear All" button gated behind a confirmation modal so a stray click can't wipe the whole list by accident.
- `?section=saved`: A deep link (used by the navbar's profile dropdown) that opens straight to the Saved Theses tab instead of Overview.

frontend/src/pages/SettingsPage.jsx — "Update your table reservation card" —■

- **Lines 1–12 (comment):** Editable fields (bio, department, research interests, profile photo) persist to user-scoped `localStorage` — again explicitly flagged as a Phase-1 placeholder for a future backend-backed version. Email is read-only (the institution controls it); "change password" is just a link into the existing Forgot Password flow rather than a duplicate password form.
- **Avatar upload pipeline (handlePhotoChange, lines 69–106):** Checks file type (PNG/JPG/WEBP) and size (≤ 4 MB) first, then draws the image onto an in-memory `<canvas>` sized down to at most 256×256 and re-encodes it as a JPEG at 85% quality — this keeps whatever gets stored in `localStorage` small regardless of how large the original photo was (a phone photo can easily be 10+ MB; this shrinks it to a few dozen KB).
- **pendingAvatar (lines 43–50):** A three-state value — undefined (no change), `null` (user clicked Remove), or a new data-URL string — so the Settings page can preview a change **before** the user clicks Save without ever touching the actually-saved avatar until they commit.
- **handleSave:** Writes the profile fields and any pending avatar change to `localStorage`, shows a success toast, and redirects to `/profile` after one second so the user can see their own confirmation message before landing on the updated profile.

Part 7 — Frontend shared components (the reusable furniture)

frontend/src/components/brand/ThesysLogo.jsx — "The restaurant's sign-painter" ■

- ****Why it draws the logo with CSS background-image instead of a normal**

`<img>` tag (lines 9–13):** the SVG file uses an advanced mask/filter trick that some browsers silently refuse to run when the SVG is loaded through `<img src>` — the logo would just be invisible. Loading it as a CSS background always triggers the browser's full SVG engine, so the mask works everywhere.

- **The crop math (lines 37–57):** the source artwork is a big 1440×810 canvas

with the actual logo mark living in one small 280×280 corner of it. Rather than pre-cropping a separate image file, this component scales the **whole** canvas up/down and shifts it so only that corner peeks through a `size×size` window — like sliding a picture frame around a big poster until only the logo shows through the hole.

- **Three variants:** `symbol` (mark only), `wordmark` (mark + "THESYS+" text, used in navbars), `full` (mark + text + "Pampanga State University · College of Computing Studies," used in a couple of full branding moments).

frontend/src/components/brand/AuthBranding.jsx — "The welcome sign above the counter" ■

A small, reused header block: the logo symbol, the "THESYS+" wordmark, and an optional uppercase subtitle line (e.g. "Sign in to continue"). Every auth page (`SignInPage`, `ForgotPasswordPage`, `RequestAccessPage`, `SetupAccountPage`, `VerifyEmailPage`, `ResetPasswordPage`) renders this at the top so the branding never has to be rebuilt page by page.

frontend/src/components/auth/ — sign-in and request forms

- `index.js` is a **barrel file** — it just re-exports five components from one place so other files can write `import { SignInCard } from '../components/auth'` instead of five separate long import paths.
- `SignInCard.jsx` — the actual sign-in form used by `SignInPage`: email + password + show/hide-password eye icon + "remember me" checkbox + submit. Validates the email looks institutional **before** ever contacting the server.
- `RequestAccessForm.jsx` — the form behind `RequestAccessPage`: name fields, a student-number-style email pattern (`digits@pampangastateu.edu.ph` — matches the backend's own `is_student_number_email` rule), a student/faculty radio choice, a drag-free "click to upload" ID document picker (PNG/JPG/PDF, ≤ 10 MB, checked client-side before ever uploading), and a Terms/Privacy agreement checkbox that opens `LegalModal` rather than navigating away.
- `ForgotPasswordForm.jsx` and `ResetPasswordForm.jsx` —

■■ **Audit finding — orphaned components.** These two files, plus `SsoButton.jsx`, are exported from `index.js` but **not imported by any page** — `ForgotPasswordPage.jsx` and `ResetPasswordPage.jsx` each build their own inline form directly instead of using these, and `SignInCard` never renders an SSO button. They still work (and are exercised by their own file, if anything imported them), they're just dead code left over from an earlier version of the flow. Worth deleting in a future cleanup pass to avoid confusing a future reader into thinking they're the live forms.
- `SsoButton.jsx` — a permanently-disabled "Sign in with SSO" button with a "Coming Soon" tooltip, built ahead of a future real school-login integration. Never actually rendered anywhere yet (see finding above).

frontend/src/components/layout/ — the shared chrome

- `AuthLayout.jsx` — covered already in the login-flow parts above (shared

header + background for all /sign-in-style pages). Not re-explained here.

- `AppNavbar.jsx` — the real navbar for every signed-in page (Repository, Thesis Detail, Title Similarity, Trend Analysis, Analytics, Profile, Settings). Desktop: logo · centered nav links · Upload button + avatar + theme toggle. Mobile: hamburger + slide-in drawer (rendered via `createPortal` so it always sits on top of everything, regardless of where in the page tree the navbar itself lives). Also exports `AvatarDropdown` separately so `LandingPage` can reuse the exact same profile menu even though it has its own custom navbar. The dropdown's **Sign Out** button doesn't sign out immediately — it opens a confirmation modal first, so a stray click can't accidentally end your session.
- `PageShell.jsx` — a tiny wrapper enforcing the **same** horizontal padding and max-width on every authenticated page, so switching tabs never causes the content to visibly shift left/right or change width.
- `PageHeader.jsx` — enforces one consistent H1 style (size, weight, color, spacing) across every page, while still letting each page pass its own custom subtitle content as children.
- `MainLayout.jsx` and `ThemeToggle.jsx` —

■ ■ **Audit finding — orphaned layout.** `MainLayout.jsx` is not referenced by any route in `App.jsx` (the actual auth-page wrapper in use is `AuthLayout`, a different file) — and `ThemeToggle.jsx` is only ever imported **by** `MainLayout.jsx`, so both are effectively dead code today. `App.jsx`'s own top comment even still says *"All auth routes remain nested under MainLayout"* (line 5) — that line is stale; they're actually nested under `AuthLayout`. Harmless (nothing calls them), but worth correcting or removing so future readers aren't misled by the comment.

frontend/src/components/legal/LegalModal.jsx — "The fine print, in a pop-up" ■

One shared modal component serving four different sets of content (About, Privacy, Terms, Help), selected by a type prop. `getContent(type)` is just a big lookup returning `{ title, body }` JSX for whichever one was asked for — `LandingPage`'s footer links and `RequestAccessForm`'s Terms/Privacy links both open this same component. It traps focus while open (`useFocusTrap`), closes on Escape or backdrop click, and locks page scroll while visible so the background page doesn't scroll underneath it.

■ ■ **Audit finding (fixed during this audit).** The Help content's *"How do I set my password?"* answer (inside the `help` branch of `getContent`) used to say: *"After verifying your email, you will receive a **second email** with a link to set your password."* — the exact same stale claim found in `RequestAccessPage.jsx`, both corrected together so the in-app help text now matches the real single-email behaviour.

frontend/src/components/landing/index.js — an empty placeholder

Just export `{}`; — a barrel file reserved for future landing-page sub-components that hasn't needed anything in it yet. Importing from it does nothing (and nothing currently does).

frontend/src/components/upload/UploadThesisModal.jsx — "Submit a new recipe to the cookbook" ■

The **global** upload form — reachable from an "Upload Thesis" button on almost any authenticated page via the shared `useUploadModal()` context (Part 9), rather than living on its own separate route.

- **UPLOAD_STAGES (lines 38–44):** A **purely cosmetic** simulated progress sequence ("Preparing upload...", "Reading document...", ...) that advances on timers while the real upload request is in flight — it doesn't reflect real server progress, it just keeps the user reassured that something is happening during what can be a several-second AI text-extraction step.
- **resetForm + the isOpen effect (lines 139–149):** Every time the modal

closes, all fields are wiped, so reopening it later always starts from a clean sheet rather than showing whatever was left over from last time.

- **handleSubmit (lines 197–266):** Client-side checks first (file attached? at least one author? at least one keyword?), then `POST /theses/upload/` as `multipart/form-data`. On success it calls `clearAllCaches()` (Part 9) so the Repository/Analytics/Trends pages don't keep showing stale cached counts that no longer include the new thesis. The error handling specifically understands `DUPLICATE_FILE` (a matching SHA-256 hash was already uploaded), `FILE_TOO_LARGE`, `FILE_TYPE_NOT_ALLOWED`, and per-field `VALIDATION_ERROR` details, mapping each to the right inline message.
- **Success screen:** Different headline depending on whether the thesis was auto-approved (faculty/admin uploads publish immediately) or sent for review (student uploads) — with buttons to view the new thesis, browse the repository, or upload another one right away.

frontend/src/components/pdf/WatermarkOverlay.jsx — "The invisible ink stamp" ■

The shared watermark used by both `ThesisDetailPage`'s (now-removed) inline preview history and `PdfPreviewPage`'s full-screen viewer. It tiles the text `"College of Computing Studies - Pampanga State University • Preview Only"` across a 9×3 grid, rotates the whole grid `-30°`, and renders it at 15% opacity so it's visible without fighting for attention against the actual document text. `pointer-events-none + select-none` mean a reader can click through it and can't accidentally select/copy the watermark text instead of the real document.

Part 8 — Frontend UI kit (the drawer of reusable utensils)

frontend/src/components/ui/ — hand-written primitives

These are THESYS+'s own small, hand-rolled building blocks — plain wrappers around a native HTML element with THESYS+'s look baked in, so every button/ input/checkbox in the app looks the same without repeating the same Tailwind classes everywhere.

- `Button.jsx`, `Input.jsx`, `Checkbox.jsx`, `Alert.jsx` — each is a tiny ~10-line wrapper: take whatever props you'd give the native element, add THESYS+'s standard styling classes, forward everything else through. The comment on each literally says `"Foundation Phase shell"` — an early, intentionally simple version.
- `Icon.jsx` — two hand-drawn SVG icons, `EyeIcon` and `EyeOffIcon`, used by the password show/hide toggles in `SignInCard` and `ResetPasswordForm`.
- `Spinner.jsx` — described already back in Part 6/earlier sections: a dual-ring animated SVG loading indicator with `sm/md/lg` sizes, used everywhere something is loading.
- `FileDropzone.jsx` — a genuinely reusable drag-and-drop **and** click-to-browse file picker used by `TitleSimilarityPage` and `UploadThesisModal`. Validates file extension and size *before* ever staging the file (rejecting bad files with a toast instead of silently accepting them), shows a "staged file" chip with a remove button once a valid file is picked, and is fully keyboard-operable (`Enter/Space` opens the native file picker, matching what a mouse click does).
- `SimilaritySlider.jsx` — the colored 30–95% threshold slider used on `RepositoryPage`. Its three "zones" (`Strict ≥ 70%`, `Balanced 50–69%`, `Broad < 50%`) each get their own pill color and a matching gradient fill on the track itself, so the slider visually communicates "how strict" without the user needing to read the number.

- `Toast.jsx` — the actual **rendering** half of the app's global notification system (the **logic** half is `ToastContext`, Part 9). Renders each toast with an icon/color/aria- live politeness matching its type (success = polite green, error = assertive red, info = polite blue) via a `createPortal` straight to `document.body`, so a toast always floats above every other layer of the page regardless of where the code that triggered it lives.

- `Logo.jsx`, `index.js` —

■ ■ **Audit finding — another orphaned component.** `Logo.jsx` (a simple gradient-text "THESYS+" wordmark) is exported from `ui/index.js` but never actually imported by any page or component — every real logo usage in the app goes through `components/brand/ThesysLogo.jsx` instead, which renders the **actual** approved brand SVG rather than plain gradient text. Same pattern as `ForgotPasswordForm/ResetPasswordForm/SsoButton/MainLayout` above: safe, unused, and a good candidate to delete in a future cleanup pass.

frontend/src/components/shadcn/ — the vendor toolbox

Thirteen files, all following the exact same recipe, so they're explained together rather than one at a time: each wraps a **Radix UI** primitive (an unstyled, fully-accessible interaction engine — handles keyboard navigation, focus trapping, ARIA attributes, portal rendering, etc.) and layers THESYS+'s Tailwind classes on top via a small `cn()` helper (see `lib/utils.js`, Part 9) that merges class name strings safely. This is the standard `[shadcn/ui]`(<https://ui.shadcn.com>) pattern — copy the component source directly into the project (rather than installing it as an opaque npm package) so it can be freely customized later.

- `alert.jsx` — `Alert / AlertTitle / AlertDescription`, with a `cva` (class-variance-authority) variant prop (default / destructive).
- `avatar.jsx` — `Avatar / AvatarImage / AvatarFallback` (the profile picture — falls back to initials if no image loads). Used by `AppNavbar's AvatarDropdown` and `ProfilePage's Sidebar`.
- `badge.jsx` — small pill labels (default / secondary / destructive / outline variants). Used throughout `Repository`, `Title Similarity`, and `Trend Analysis` for status/score/keyword chips.
- `button.jsx` — a more feature-complete button than `ui/Button.jsx`, with 6 visual variants and 4 sizes, plus an `asChild` prop (via Radix's `Slot`) that lets the button's styling apply to a **different** element (e.g. a `<Link>`) instead of rendering an actual `<button>`.
- `card.jsx` — `Card / CardHeader / CardTitle / CardDescription / CardContent / CardFooter` — a generic bordered container family.
- `dialog.jsx` — a full modal dialog primitive (overlay + centered panel + built-in close button + open/close animations) built on Radix's `Dialog`. Not currently used by any custom app modal (`LegalModal`, `UploadThesisModal`, and the sign-out confirmation each hand-roll their own `createPortal`-based modal instead), but available for future use.
- `dropdown-menu.jsx` — a full dropdown-menu primitive family (trigger, content, items, checkbox items, radio items, submenus, keyboard shortcuts hint). Like `dialog.jsx`, not currently wired into any page — `AppNavbar's` own profile dropdown is hand-built instead.
- `select.jsx` — a styled `<select>` replacement built on Radix `Select`, with a scrollable popover, up/down scroll buttons, and a checkmark next to the selected item. Not currently used — the app's actual dropdowns (`Repository's` year/program filters, the upload form's `Program` select)

use plain native `<select>` elements instead.

- `separator.jsx` — a one-pixel divider line (horizontal or vertical).
- `skeleton.jsx` — a pulsing gray placeholder block — conceptually the same idea as the hand-rolled `.thesys-skeleton` CSS class used throughout Repository/Analytics/Trends loading states, just packaged as a component instead of a utility class.
- `slider.jsx` — a generic Radix-based range slider. Not used directly — `SimilaritySlider.jsx` implements its own slider with custom zone coloring instead of building on this one.
- `tabs.jsx` — a full accessible tabs primitive (list/trigger/content).
Not currently used — `ProfilePage`'s Overview/Saved Theses tabs are hand-built with plain buttons and conditional rendering instead.
- `tooltip.jsx` — `Tooltip / TooltipTrigger / TooltipContent` + a `TooltipProvider` that must wrap the app once (done in `App.jsx`). This one **is** actively used — by `RepositoryPage`'s semantic-match badges and similarity-slider helper text, and by `AnalyticsDashboardPage`'s stat-card tooltips.
- `index.js` — the barrel file re-exporting all of the above from one import path, explicitly separated from `components/ui/` (see its own top comment) specifically so a `Button` from one folder is never accidentally confused with the differently-styled `Button` from the other.

■ **Note on the unused ones:** `dialog.jsx`, `dropdown-menu.jsx`, `select.jsx`, `slider.jsx`, and `tabs.jsx` are legitimately unused today, but unlike the earlier "orphaned component" findings, these were installed as a **toolbox for future screens** (that's the whole point of the `shadcn/ui` copy-paste pattern — you install the primitive once, then use it whenever a future feature needs it) rather than being **leftover** from an earlier version of an existing feature. Not a defect — just worth knowing which primitives are "available" versus "in active use" today.

Part 9 — Frontend context, hooks, and utilities (the wiring behind the walls)

frontend/src/context/ThemeContext.jsx — "The light switch that remembers" ■

- `getInitialTheme` (lines 18–27): On first load, checks `localStorage` for a previously saved choice; if there isn't one, asks the **operating system** whether it prefers dark mode (`prefers-color-scheme: dark`) and matches that; only falls back to light as a last resort.
- **The provider's effect** (lines 32–40): Whenever theme changes, it adds or removes the `dark` class on the root `<html>` element (which is what makes every `dark: Tailwind` class in the whole app activate or deactivate) and saves the new choice back to `localStorage` under `thesys.theme` so it persists across visits.
- `useTheme()`: Throws a clear error if used outside `<ThemeProvider>` — much easier to debug than a silent undefined crash somewhere else.

frontend/src/context/ToastContext.jsx — "The little bell the kitchen rings" ■

The **logic** half of the toast notification system (`components/ui/Toast.jsx` is the **visual** half).

- `push` (lines 42–53): Adds a new toast to the list, capped at

`MAX_TOASTS = 3` — if a fourth arrives, the **oldest** one is dropped so the screen never fills up with a huge stack, and schedules its own auto-dismissal after 4 seconds.

- `dismiss`: Removes a toast immediately (used both by the auto-dismiss timer and by the toast's own close button) and clears its pending timer so a manually-closed toast doesn't try to dismiss itself again later.
- Exposes just `toast.success(msg) / toast.error(msg) / toast.info(msg)` — three tiny convenience methods so calling code never has to think about IDs, timers, or the underlying array.

frontend/src/context/UploadModalContext.jsx — "The kitchen's order slip slot" ■

A very small context: just `isOpen / open() / close()`. This exists so *any* "Upload Thesis" button anywhere in the app (navbar, landing page, mobile drawer) can open the **same single** `UploadThesisModal` instance mounted once at the app root, instead of every page needing its own copy of the upload form.

frontend/src/hooks/useFocusTrap.js — "Keep your hands inside the ride" ■

An accessibility hook used by every modal/drawer in the app (`LegalModal`, the mobile nav drawers, the sign-out confirmation, `UploadThesisModal`). While a dialog is open, pressing `Tab` should only cycle through the *elements inside that dialog* — not "leak" focus out to buttons on the page behind it, which is confusing for keyboard and screen-reader users. On open, it remembers whichever element was focused beforehand; while open, `Tab/Shift+Tab` wrap around from the last focusable element back to the first (and vice versa); on close, it gives focus back to whatever triggered the dialog in the first place, so the keyboard cursor lands back where the user expects it.

frontend/src/hooks/useProfilePicture.js — "The photo on your name badge" ■■

Reads/writes the profile photo (a base64 data URL) from user-scoped `localStorage`. The interesting bit is the custom `AVATAR_EVENT` (lines 22–41): if `SettingsPage` saves a new photo while `AppNavbar` is *also* mounted on the same page, the navbar's own copy of this hook needs to know about the change immediately, but they're two separate hook instances with no direct connection to each other. So `save()` writes to `localStorage` *and* fires a plain browser window event; every other mounted instance of this hook listens for that event and re-reads the new value — a lightweight way to keep multiple components in sync without a heavier state-management library.

frontend/src/hooks/useToast.js / useUploadModal.js — thin context accessors

Both are tiny (~10-line) hooks that just read their respective context and throw a clear error if used outside the matching Provider — the same pattern as `useAuth` in Part 2.

frontend/src/utils/appCaches.js — "Wipe the whiteboard before the next customer" ■

Several pages (`RepositoryPage`, `TrendAnalysisPage`, `AnalyticsDashboardPage`) keep their own **module-level** in-memory cache to avoid re-fetching the same data on every visit. The problem: if User A signs out and User B signs in on the *same browser tab*, those caches are just sitting in memory — User B would briefly see User A's cached search results. This tiny file solves that with a simple **registry** pattern: each page's cache module calls `registerCacheClearer(fn)` once, at load time, handing over "here's how to empty *my* cache." `AuthContext.signOut()` (and `UploadThesisModal` after a successful upload) then just call `clearAllCaches()`, which loops through every registered clearer — neither of those callers needs to know which pages exist or what their caches look like.

frontend/src/utils/userStorage.js — "Everyone's own labeled locker" ■

The foundation underneath saved theses, profile bio/photo, and every other piece of "per-user" data that (in this phase) lives in the browser instead of the database. `getUserKey` builds a key like `thesys.savedTheses.<userId>` — scoped to the signed-in user's ID (or email, as a fallback) — so two different people signing into the same browser never see each other's saved theses or profile edits. `getUserData/setUserData/removeUserData` wrap `localStorage.getItem/setItem/removeItem` with automatic JSON encoding/decoding and defensive `try/catch` blocks (a full or disabled `localStorage` should degrade gracefully, never crash the page).

frontend/src/lib/Utils.js — the `cn()` helper

A two-line utility every `shadcn` component imports: `clsx` merges conditional class name strings/objects into one string, and `tailwind-merge` then resolves any *conflicting* Tailwind utilities in that string (e.g. if both `px-2` and `px-4` end up in the same class list, the later one wins instead of both being applied and the browser picking arbitrarily).

frontend/src/api/index.js — a one-line re-export

`export { default as client } from './client';` — lets other files write `import { client } from '../api'` as an alternative to importing `client.js` directly. Purely a convenience alias.

frontend/src/styles/tokens.js — the design system's single source of truth

Maps semantic names (`canvas`, `ink`, `muted`, `success-bg`, ...) to CSS custom properties defined in a separate `tokens.css` file, and feeds `CHART_PALETTE` — the exact 8-color sequence used by every inline SVG chart in the app (Trend Analysis's doughnut and bar charts, Analytics' bar charts) — from one place, so changing the brand's chart colors means editing this one array instead of hunting through every chart component. The comment explains the underlying trick: because both the light (`:root`) and dark (`.dark`) CSS selectors define the *same* variable name with different values, a single Tailwind class like `bg-canvas` automatically resolves to the right color in whichever theme is active — no `isDark ? '...' : '...'` ternary needed for colors that go through this system.

Part 10 — Backend: the accounts app (the master member ledger) ■

Every other backend app refers back to this one — it owns the single `User` table that the whole restaurant's staff and customers are listed in.

backend/accounts/models.py — "The one big member ledger"

- **Role (lines 28–44):** Exactly three roles exist — `student/faculty/administrator` — stored as plain lowercase text, never as a separate lookup table. `Role.ALLOWED_FOR_REQUEST` is bolted on after the class body (a `TextChoices` is a real Python Enum, so you can't just add a tuple as a normal class attribute inside it) and deliberately **excludes** `administrator` — nobody can request to become an administrator through the public sign-up form; that role only gets assigned by another administrator, directly, through Django admin.
- **UserManager.create_user (lines 52–90):** The one place new accounts get created. Notably accepts `password=None` — this is exactly what lets the access-approval flow create a fully real account (with a real email, name, and role) **before** the person has ever chosen a password. Rather than using Django's usual "unusable password" sentinel string, it writes a literal database `NULL`, so later code can tell "no password yet" apart from "has a weird/corrupted password hash" with a simple truthiness check.
- **UserManager.create_superuser (lines 92–122):** Used by `manage.py`

createsuperuser. Forces the role to administrator, is_staff=True, and is_superuser=True` so Django's own admin panel login works, and marks the email pre-verified (superusers skip the whole access-request pipeline entirely — they're created directly on the server).

- **User (lines 125–203):** The single users table (`db_table = 'users'`).

Two details worth calling out: - The password **field is deliberately nullable** (line 146) — Django normally requires it, but THESYS+ overrides it specifically to support the "account exists, password not set yet" state described above. - `last_login_at` **vs. Django's built-in** `last_login`: two **different** timestamps exist on purpose. `last_login` is Django's own field, updated by its session-login machinery (which THESYS+ doesn't really use, since it's a JWT API); `last_login_at` is the one the actual `LoginView` updates, and is the one that means something in this system. - `save()` **override (lines 196–202):** Belt-and-suspenders — the database column is already case-insensitive (`CIEmailField/citext`), but this also lowercases the email in Python before saving, so what's **displayed** is always consistently lowercase too.

backend/accounts/permissions.py — "Who's allowed behind the counter?"

- **PermissionRegistry (lines 18–36):** A simple in-memory dictionary

mapping each role to a **set** of permission-key strings (e.g. `auth.read_self`). Each Django app can add its own keys to a role's set from its own `AppConfig.ready()` — so theses can grant theses.upload to faculty without ever having to edit this file or the accounts app at all. `GET /auth/me/` reads from this registry to tell the frontend exactly what the signed-in user is allowed to do.

- **IsAdministrator (lines 39–60):** A DRF permission class used to guard

every admin-only endpoint (approving/denying access requests, etc.). It deliberately distinguishes two failure cases: **not logged in at all** returns a plain `False` (DRF turns that into a generic 401), while **logged in but wrong role** **raises** a custom `InsufficientRole` exception directly — the comment explains this is needed because DRF's default `PermissionDenied` would otherwise overwrite the more specific `INSUFFICIENT_ROLE` error code the frontend actually checks for.

backend/accounts/services.py — "The password-checking clerk, for hire by other departments"

Two small cross-app functions so `auth_service` and `password_reset` never have to poke at password-hashing internals themselves:

- `set_password`: Hashes with whatever the **first** entry in `settings.PASSWORD_HASHERS` is (Argon2id — see Part 17) and saves.
- `verify_password (lines 32–58)`: Returns **two** booleans, not just one — ok (did the password match?) and `needs_rehash` (is the **stored** hash using an older/weaker algorithm than the current default?). This lets `LoginView` silently upgrade a user's password hash to the current best algorithm the moment they successfully log in, without ever forcing them to change their actual password — a standard, unnoticeable security hygiene practice for password-hashing schemes that improve over time.

backend/accounts/admin.py / urls.py / apps.py — the supporting cast

- `admin.py` registers `User` with Django's built-in admin site

(extending its stock `UserAdmin`), exposing email/name/role/status columns with search and filters, and marking audit fields (`id`, timestamps) as read-only so nobody can hand-edit them by mistake through the admin UI.

- `urls.py` is intentionally **empty** — a placeholder so the URL mount

point exists, but this app exposes no HTTP endpoints of its own (login, logout, etc. all live in `auth_service` instead).

- `apps.py`'s `ready()` seeds the permission registry above with one

baseline permission (`auth.read_self`) for all three roles, so every signed-in user — regardless of role — is at minimum allowed to read their own profile.

Part 11 — Backend: the `access_requests` app (the front-door registration desk) ■

This app owns the *entire* "I'm not a member yet, please let me in" journey — from the very first form submission through document verification, email proof, and finally account creation. This is also where the **single-email onboarding fix** (Spec: "Eliminate Second Email and Enable Inline Password Setup") from earlier in this project actually lives.

backend/access_requests/models.py — "The registration desk's paper trail"

- `AccessRequest` (lines 29–109): One row per person who has ever asked

for access. `status` has seven possible values — walking through them tells the whole story: `pending` (legacy manual-review path, or a document that needs a human), `processing` (the AI verification pipeline is actively looking at the uploaded ID), `auto_approved/pending_manual_review/denied` (the pipeline's three possible verdicts), `pending_email_verification` (clean auto-approval — waiting for the applicant to click the emailed link), and finally `approved` (the account has been created). - The **partial unique index** on line 101–105 (`UniqueConstraint(..., condition=Q(status='pending'))`) is a database-level guarantee: *at the database itself*, not just in application code, two *pending* requests for the same email can never both exist — so even a network glitch that causes a double-submit can't create duplicate pending requests. - `requested_role` is restricted by its own `CheckConstraint` to `student/faculty` — the database itself refuses an `administrator` row here, as a second layer of defense behind the serializer's own check.

- `EmailVerificationToken` (lines 112–154): A single-use, 24-hour token

tied to one `AccessRequest`. The docstring spells out the **current** (post-fix) flow precisely: clicking the emailed link calls `approve_request(..., send_email=False)` — creating the `User` **and** issuing a setup-password token, but explicitly **not** emailing that second token. Its plaintext travels back up through the view's JSON response instead, straight to the browser that's already open on the verification page — which is exactly how the second email got eliminated.

backend/access_requests/serializers.py — "The registration form itself"

- `RequestAccessSerializer` (lines 30–136): Supports **two** different

submission shapes **at once** — the original free-text justification flow, and the newer document (ID photo/PDF) upload flow — and figures out which one is being used by checking whether a document was attached. - `validate_email` (lines 56–77): **Applies a stricter** email pattern for the document-upload flow (must be exactly `<digits>@pampangastateu.edu.ph` — a real student-number format) versus the legacy flow (any `*.pampangastateu.edu.ph` address is fine). This matters because the document-upload flow is meant specifically for students proving themselves via a Student ID, which always carries a student number. - `validate_requested_role`: Re-enforces (at the serializer layer, in addition to the database constraint above) that only `student/faculty` can be requested — `Role.ALLOWED_FOR_REQUEST` from `accounts/models.py` is the single source of truth both places read from. - `validate_document` (lines 89–115): An early, cheap check (file size, file extension) ***before*** the much more expensive OCR/AI verification pipeline ever runs — no point spending seconds of OCR time on

a file that was always going to be rejected for being a .exe or 40 MB. - validate (**lines 117–136**): Cross-field rule — you must supply *either* justification *or* document, never both, never neither.

- ****AccessRequestListItemSerializer / AdminApproveSerializer /**

AdminDenySerializer: Shape the admin-facing list/approve/deny endpoints. Denial *requires* a non-blank reason (line 173) — the applicant's rejection email needs to say *something* actionable, not just "no."

backend/access_requests/email_verification.py — "Prove the email is really yours, then walk right in"

This module is the heart of the auto-approval path for the document-upload flow, and the exact place the single-email fix was implemented.

- **issue_email_verification_token (lines 54–121):** Called right after the

AI pipeline decides a document looks clean. Flips the request to `pending_email_verification`, creates a 24-hour `EmailVerificationToken`, and sends **the first (and, now, only) email** — subject "Verify your THESYS+ email address" — containing a link to `/verify-email?token=...`

- **consume_email_verification_token (lines 124–200):** Called when the

applicant clicks that link. 1. Look up the token by its SHA-256 hash (locking the row so two simultaneous clicks on the same link can't race each other), reject if missing/expired/already-used. 2. Temporarily flip the request's status to `processing` so `approve_request` (below) will accept it. 3. Call `approve_request(req, reviewer=None, note='Account activated via email verification.', send_email=False)` — **the `send_email=False` is the actual fix**. It creates the `User` row and issues a setup-password token, but the token is never emailed — its plaintext comes back on the `ApprovalOutcome` object returned by this function. 4. Mark the verification token as used. - The docstring at the top of the file (lines 1–16) explicitly documents this contract for future readers, so nobody accidentally "fixes" it back to sending a second email by mistake.

backend/access_requests/services.py — "The actual decision-maker behind the counter"

- **approve_request (lines 43–119):** The single function both the

manual admin-approval path (`ApproveAccessRequestView`) and the

auto-approval-via-email-verification path (above) both call — one shared, carefully-audited approval routine instead of two separate half-copies of the same security-sensitive logic. Inside one

`transaction.atomic()` block: 1. Row-lock the `AccessRequest` (`select_for_update()`) so two concurrent approval attempts can't both succeed. 2. Assert it's actually still `pending/processing` — otherwise raise `AccessRequestNotPending`. 3. Create the `User` (password `None` — see Part 10) via `accounts.UserManager.create_user`, catching `IntegrityError` (a race where the email got

registered by something else in the meantime) as `AccessRequestEmailCollision`. 4. Stamp the request approved. 5. Call `password_reset.services.issue_reset_token(user,`

`template='account_activation', send_email=send_email)` — this is where the **new**

`send_email` **parameter** (added for the single-email fix) gets threaded through. The **default** stays `True`, so the ordinary admin-approval path (where there's no other communication channel with the applicant) still sends its one "Welcome to THESYS+ - Set Up Your Account" email exactly as before — **only** the

email-verification path explicitly passes `send_email=False`. - `ApprovalOutcome` (the return value) now carries an optional `reset_token_plaintext` field, populated *only* when `send_email=False` — a deliberate design choice so the plaintext token is never accidentally exposed anywhere the email-sending path is used, only where the caller has explicitly said "I'm handling delivery myself." - The ****audit-log write** happens *outside* the transaction** (line 104 onward) — a defensive choice so that if writing the audit row

itself fails for some unrelated reason, it can never roll back the user creation that already succeeded.

- `deny_request` (**lines 127–184**): Simpler — row-lock, assert pending, stamp denied with the reason, then send the denial email **outside** the transaction (a failed email shouldn't undo an already-committed denial) and write the audit entry.

backend/access_requests/views.py — "The registration desk's four counters"

- `RequestAccessView` (`POST /request-access/`): Branches into `_handle_legacy_justification` (simple: create a pending row, done) or `_handle_document_upload` (the interesting one): validates the file, hashes it (SHA-256) and saves it to **private** storage (`media/private/...`, `chmod 0o700` — not publicly served), runs the AI verification pipeline (`identity_verification`, Part 12) synchronously, and routes the result: `auto_approved` → send the verification email (above); `pending_manual_review` → leave it for a human; `rejected` → already marked denied by the pipeline itself. Any pipeline exception is caught, logged as a `VerificationResult` with `status='error'`, and the request falls back to pending (manual review) rather than silently failing — a broken AI model should never mean a legitimate applicant's request just vanishes.
- `VerifyEmailView` (`GET /verify-email/?token=...`): Calls `consume_email_verification_token` and returns `{ verified, email, setup_token, message }` — that `setup_token` field is the plaintext the frontend's `VerifyEmailPage` uses to immediately call `POST /auth/setup-password/` on the **same page**, without a second email ever being dispatched.
- `**AccessRequestListView / ApproveAccessRequestView / DenyAccessRequestView**` The three admin-only counters (all behind `IsAdministrator`), listing/approving/denying pending requests. Approval here uses `approve_request(..., send_email=True)` by default — the single "Welcome to THESYS+ - Set Up Your Account" email, unchanged by the fix.

backend/access_requests/urls.py / urls_admin.py / urls_public.py / apps.py

- `urls.py` is an empty placeholder — the *actual* routes live in the two files below, which is a slightly unusual split worth noting: the app has **two separate URL modules**, one mounted under `/api/v1/auth/` (public — `request-access/`, `verify-email/`) and one under `/api/v1/admin/` (staff-only — the `list/approve/deny` endpoints), so the root `URLConf` can apply different prefixes/permissions to each without the app needing its own internal namespacing logic.
- `apps.py` is a minimal `AppConfig` with no `ready()` hook — this app doesn't register any permissions of its own (approval/denial power comes from `accounts.IsAdministrator`, not a custom permission key).

Part 12 — Backend: the `identity_verification` app (the AI bouncer that checks your ID) ■■

When someone uploads a Student ID or Certificate of Registration during `RequestAccessForm`, **this** app is what actually looks at the photo and decides whether it's really a genuine `PampangaStateU/CCS` document — entirely automatically, most of the time, without waking up a human administrator.

The five-step pipeline, in plain language

Think of this like airport security scanning a passport:

1. `file_validator.py` — **"Is this even a real photo/PDF, and is it safe?"** Checks the file's true type by reading its first few bytes (**magic bytes**) rather than trusting the filename extension (a file named `id.png` that's secretly something else gets caught here). Then it **sanitizes** the file: images are re-encoded through Pillow (which silently strips hidden EXIF metadata — GPS coordinates, camera model, etc. — that a photo can carry without the uploader realizing), and PDFs are re-saved via `pikepdf` with any embedded JavaScript or hidden embedded files stripped out. This runs **before** anything expensive (OCR) so a bad/malicious upload never even reaches later stages.

2. `services/ocr_extractor.py` — **"Read the text off the photo."** Uses **Tesseract OCR** (`pytesseract`) to turn the picture into plain text, plus a **confidence score** (Tesseract's own per-word certainty, averaged). PDFs are first converted to an image of their first page (`pdf2image`) at high resolution (300 DPI) before OCR runs on it, since Tesseract only reads images, not PDF structure. `_configure_tesseract_path` (lines 22–68) is a small but important cross-platform fix: on Windows, Tesseract often isn't on the system PATH even when installed, so this function auto-discovers it in the common install locations rather than crashing with "tesseract not found."

3. `services/field_extractor.py` — **"Pull out the name, school, and program from that raw text."** A big collection of regex patterns tries labeled fields first ("Name: Juan De la Cruz"), then falls back to **unlabeled heuristics** for IDs that don't use clean labels at all — e.g. scanning for lines that are "2–4 uppercase words, not a known keyword like UNIVERSITY or BACHELOR" as a plausible name, or detecting a university name split across two OCR lines. This fallback logic exists because real Student IDs vary enormously in layout, and OCR itself introduces typos/gaps that a single strict pattern would miss. `ProgramNormalizer` (`normalizers.py`) then maps whatever variant was found ("BSIT", "BS-IT", "bs information technology", ...) to one of the four canonical program names.

4. `validators/rule_validator.py` — **"Does this match what a real CCS PSU document should say?"** Checks the extracted institution, college, and program against allow-lists, with a clever two-pass approach: ***negative detection*** first (does the raw OCR text obviously mention a **different** university, like "University of the Philippines"? — that's an unambiguous rejection, not just a missing field), then fuzzy matching for the real thing (`rapidfuzz.fuzz.token_set_ratio` tolerates OCR mangling like "Ventur Hs Tate University" still matching "Don Honorio Ventura State University" at a 75%+ similarity threshold). It also **infers the college** from the program alone when the college field is missing — real PSU Student IDs typically print only the program (e.g. "BS Computer Science"), never the college name, so if the program is one of CCS's four canonical programs, "College of Computing Studies" is assumed rather than flagged as a missing field.

5. `services/decision_engine.py` — **"Given all of that, what's the verdict?"** This is the actual decision tree, and it's worth spelling out because it's the crux of the whole "who gets in automatically" logic: - Rules failed with a **clear** wrong institution/college/program → **REJECTED**. - Rules failed only because fields were **missing** → check the raw OCR text itself for PSU/DHVSU "signal" keywords (distinctive ones like `*pampanga*`, `*ventura*`, `*dhvsu*` need only appear once; weaker ones like `*psu*`/`*ccs*` need at least two together, since those abbreviations show up in unrelated contexts too). No signal at all → **REJECTED** (`invalid_institution` — probably not a PSU document at all). Signal present → **PENDING_MANUAL_REVIEW** (probably a real PSU ID that's just blurry or partially unreadable — worth a human's second look rather than an automatic rejection). - Rules **passed** → check OCR confidence: $\geq 75\%$ → **AUTO_APPROVED** (clean pass, triggers the single verification email from Part 11); 60–74% → **PENDING_MANUAL_REVIEW** (rules matched but the scan quality itself is questionable); $< 60\%$ → **PENDING_MANUAL_REVIEW** as well. - **The overall design principle:** the system is deliberately biased toward **"ask a human"** over **"reject automatically"** whenever there's genuine ambiguity — a legitimate PampangaStateU student should never get auto-rejected just because their phone photo was slightly blurry.

`services/orchestrator.py` — "The pipeline's stage manager"

`VerificationOrchestrator.verify_request` runs all five steps above, in order, inside one `@transaction.atomic` block, writing a `VerificationResult` row (OCR text, confidence, extracted fields, decision reason, any rule failures) and updating the `AccessRequest`'s own status to match (processing for auto-approved — the `*view*` layer finishes the job by calling `approve_request`; pending for manual review; denied for rejected). Any unexpected exception anywhere in the pipeline is caught, logged as `status='error'`, and the request falls back to pending rather than crashing the whole submission — a broken OCR install or a corrupt PDF should never silently swallow someone's access request.

models.py — `VerificationDocument` / `VerificationResult`

Two tables recording, respectively, **what was uploaded** (file path in **private** storage, MIME type, SHA-256 hash for anti-replay/duplicate detection, size) and **what the pipeline concluded** (status, the OCR raw text and confidence, the structured extracted fields as JSON, the decision reason, and any specific rule failures) — kept as separate rows from `AccessRequest` itself so the potentially-large OCR text/extracted-fields blob doesn't bloat every query against the request table.

constants.py* / *types.py* / *protocols.py* / *admin.py* / *apps.py

- `constants.py` — shared error codes and the file-type allow-list (PNG/JPG/PDF, with their magic-byte signatures) used by `file_validator.py`.
- `types.py` — just re-exports the result dataclasses from their real homes (`FileValidationResult`, `OCRResult`, `ExtractedFields`) into one convenient import location.
- `protocols.py` — currently an empty placeholder (``# Protocol definitions will be added as needed``) reserved for future dependency- injection interfaces; nothing depends on it yet.
- `admin.py` registers both models with Django admin as **read-only** inspection views (every field is in `readonly_fields`) — administrators can look at what the pipeline decided and why, but can't hand-edit OCR results or extracted fields through the admin UI.
- `apps.py` is a minimal `AppConfig` with no custom `ready()` logic.

■ A handful of top-level scripts in `backend/` (`demo_rule_validator.py`, `verify_app_setup.py`, `inspect_ocr_text.py`, `diagnose_failed_extraction.py`, `demo_program_normalization.py`, `test_real_psu_id_extraction.py`, etc.) are **manual developer diagnostic scripts** — run by hand from the command line to sanity-check the OCR/rule pipeline against a real ID image while building or debugging it. They aren't imported by the running application or the test suite, so they're intentionally not covered file-by-file here.

Part 13 — Backend: the `password_reset` app (the locksmith counter)

Everywhere a plaintext password-setting link is issued and later redeemed — forgot-password recovery, admin-approval account setup, and (indirectly) the single-email inline flow — this app is the locksmith doing the actual key- cutting and key-checking.

***backend/password_reset/models.py* — "The notebook of unused keys"**

`PasswordResetToken` — one row per issued token: which User it belongs to, its **SHA-256 fingerprint** (`token_hash`, never the plaintext itself), `expires_at` (30 minutes after issuance), and `used_at` (null until redeemed). The exact same mechanics as `auth_service`'s `RefreshToken` model from Part 4 — hash-only storage, single-use, time-boxed — reused here for a different kind of secret.

backend/password_reset/services.py — "Cut a new key, or use one to open the door"

- `issue_reset_token` (lines 53–117): Generates a random opaque token, saves only its hash, and builds the URL the user will click. - `_build_reset_url` (lines 41–50): Routes the link to **one of two different pages** depending on `template` — `account_activation` links go to `/setup-account`, everything else goes to `/reset-password`. This is what lets the same underlying token mechanism serve two visually and contextually distinct pages. - **The `send_email` parameter** is the actual piece that eliminated the onboarding flow's second email: when `False`, the function does everything **except** actually calling the email backend — it still creates the token row and returns its plaintext, but the caller (`access_requests.email_verification.consume_email_verification_token`) is the one deciding to hand that plaintext straight back to the browser instead of relaying it by email.
- `consume_reset_token` (lines 130–185): The atomic redemption. Row-locks the token by its hash, checks it's neither expired nor already used, sets the new password (via `accounts.services.set_password`), flips `is_email_verified = True` (successfully setting a password sent to *your* institutional email is itself proof you control that address), marks the token consumed, and — importantly — calls `revoke_all_for_user` so **every existing login session for that user is immediately killed**. This is a deliberate security choice: if someone just reset their password (maybe because an old one leaked), any session an attacker might already be holding open gets logged out too, not just prevented from logging in again.

backend/password_reset/views.py — "Three counters at the locksmith"

- `ForgotPasswordView` (POST `/forgot-password/`): Always replies ``200`` `{"status": "ok"}`` — **whether or not** the email actually belongs to a real account — specifically to prevent an attacker from using this form to discover which email addresses have THESYS+ accounts (this is called "preventing user enumeration"). It still validates that the domain *looks* institutional (since that's public knowledge anyway, not a secret worth protecting) before doing anything.
- `ResetPasswordView` (POST `/reset-password/`): The recovery-flow counter — calls `consume_reset_token` and returns a generic `{"status": "ok"}`.
- `SetupPasswordView` (POST `/setup-password/`): The **newer**, onboarding-flow counter, added specifically for the single-email fix. Its docstring says outright that it shares `consume_reset_token` with `ResetPasswordView` — the underlying token validation/password-setting logic is *identical* — but it's exposed on its own route with its own response wording (`{"message": "Account activated successfully. You can now sign in."}`) so the frontend's setup-account page never has to reason about recovery-flow-specific copy, and vice versa. Reusing the same service function instead of writing a second copy of the same security-sensitive atomic logic was a deliberate choice to avoid two implementations quietly drifting apart over time.
- All three endpoints sit behind `require_origin_match` (only THESYS+'s own frontend may call them) and IP/token-based rate limiting (Part 16) to slow down brute-force attempts at guessing a valid token.

backend/password_reset/serializers.py / urls.py / apps.py

- `serializers.py` — `ForgotPasswordSerializer` (just an institutional email check) and `ResetPasswordSerializer` (token + new password, with the same 12-character/letter/digit strength rule enforced here that the frontend already checks client-side — the backend never trusts the frontend's own validation, since a browser's rules are trivially bypassable by

anyone calling the API directly).

- `urls.py` mounts `forgot-password/`, `reset-password/`, and `setup-password/` all under the shared `/api/v1/auth/` prefix.
 - `apps.py` is a minimal `AppConfig` — no custom `ready()` logic; this app doesn't register any permission keys of its own.
-

Part 14 — Backend: the theses app (the actual cookbook shelf, plus its AI librarian) ■■

This is the biggest app in THESYS+ — it owns the thesis repository itself **and** every AI feature built on top of it (semantic search, title similarity, topic trends, analytics), plus the watermarked-preview fix from the most recent round of work.

backend/theses/models.py — "The shelf label on every book"

Thesis — one row per uploaded document. Worth calling out:

- **sha256 is globally unique** (line 73) — the database itself refuses two theses with byte-identical file content, which is what powers the frontend's "This file has already been uploaded" duplicate check.
- `embedding_vector` stores the 384-number SBERT fingerprint as a plain JSON array rather than requiring a specialized vector database extension (`pgvector`) — a deliberate simplicity trade-off appropriate for a thesis-scale corpus; the comment notes this can be swapped for a real vector index later without touching any calling code.
- **Three** `CheckConstraints` (`status`, `file_type`, `year range`) enforce data integrity at the database level, not just in Django forms — so even a direct SQL insert or a bug in application code can't leave a row with an invalid year or an unrecognized status.

backend/theses/validators.py — "Is this really a PDF or DOCX?"

A lighter cousin of `identity_verification`'s file validator, tuned for thesis documents: 25 MB cap, PDF or DOCX only, and a magic-byte check — `%PDF-` for PDFs, and `PK\x03\x04` for DOCX (a DOCX file is secretly a ZIP archive under the hood, so its magic bytes are the same as any ZIP's).

backend/theses/serializers.py — three different "faces" of a Thesis

- `ThesisUploadSerializer` — validates the metadata half of an upload (`title`, `abstract`, `authors`, `keywords`, `program`, `year`, `adviser`); the file itself is validated separately by `validators.py` since a `FileField` doesn't fit naturally into the same shape as the rest of the form.
- `ThesisListItemSerializer` — the compact "card" shape for repository listings, with a computed `uploaded_by_name` and an optional `similarity_score` (only populated when the list came from a semantic search — plain browsing has no score to show).
- `ThesisDetailSerializer` — the full record for the single-thesis page, including a computed `download_url` that always points at `/theses/{id}/download/` — the same endpoint this project's watermarked- preview fix repurposed to serve inline PDF bytes instead of a forced download.

backend/theses/services/text_extractor.py — "Read the whole document into searchable text"

Runs once, right after a successful upload, to populate `Thesis.extracted_text` — the raw text every later AI feature (semantic search, title similarity, topic clustering) is built from.

- **PDF (`_extract_pdf`):** Tries `pypdf`'s direct text extraction first (fast, and accurate for "born-digital" PDFs with a real text layer). If that yields fewer than 200 characters — a strong signal the PDF is actually a **scanned image** with no embedded text — it falls back to running the same Tesseract OCR engine `identity_verification` uses, page by page (capped at 50 pages), reusing its Windows Tesseract path-discovery logic rather than duplicating it.
- **DOCX (`_extract_docx`):** Uses `python-docx` to flatten every paragraph and table cell into plain text (this is also the same extraction logic the new `preview_pdf.py` heading-detection has to independently re-derive from the raw paragraphs, since this function only returns flat text — see below).

backend/theses/services/semantic_search.py — "Find theses by *meaning*, not just matching words" (Phase 2A)

- **The model:** `sentence-transformers/all-MiniLM-L6-v2` — 384-dimensional embeddings, CPU-friendly (no GPU needed), loaded **once per server process** and cached at module level (`_get_model`) since loading it takes real time and memory.
- `embed_text`: Turns any string into a 384-number vector, ****L2-normalized**** — a deliberate choice that means "cosine similarity" (the standard way to compare meaning-vectors) reduces to a plain dot product, which is much cheaper to compute at scale.
- `compose_thesis_text`: Builds the canonical text fed into the model for each thesis — title + abstract + a **truncated** 2000-character slice of the extracted text + keywords. Truncating matters because the model itself has a hard input-length cap; feeding it an entire 40-page thesis would just get silently cut off anyway, so the truncation deliberately keeps the most meaningful parts (title/abstract) intact and unclipped.
- `rank_theses`: Given a search query, embeds it once, then computes a dot product against every candidate thesis's stored embedding (skipping any thesis that doesn't have one yet) — entirely in NumPy, no external vector database needed at this scale.

backend/theses/services/title_similarity.py — "Has this exact idea been done before?" (Phase 2B)

Deliberately **reuses** the same SBERT model from `semantic_search.py` (`embed_text`) but compares a candidate title against every existing thesis's **title only** — not the fuller title+abstract+text blend semantic search uses. The comment explains why: comparing against full abstracts would make a candidate title look artificially similar to any thesis that merely **mentions** related concepts in its abstract prose, producing false duplicate-topic warnings. Three classification bands (`HIGHLY_SIMILAR` $\geq 85\%$, `MODERATELY_SIMILAR` 60–84%, `LOW_SIMILARITY` below that) each carry a canned, defensible recommendation sentence — the kind of language that could be quoted in an actual thesis-defense committee meeting.

backend/theses/services/topic_analysis.py — "What is everyone researching lately?" (Phase 3)

The most involved of the three AI services: 1. **TF-IDF** (`TfidfVectorizer`) turns every approved thesis's combined text into a sparse "which words matter most, relative to the whole corpus" vector — with a

hand-curated `_EXTRA_STOP_WORDS` list (lines 156–168) that strips generic thesis-boilerplate words ("study," "proposed," "system," "Pampanga State University," ...) that would otherwise dominate every cluster's keyword list without actually describing any particular *topic*. 2. **K-Means clustering** groups those vectors into *k* topic clusters, with *k* auto-sized to the corpus (5–8 clusters normally; for a tiny corpus under 5 theses, *k* shrinks to match so every document isn't forced into an oversized bucket). 3. **Heuristic topic naming** (`_TOPIC_RULES`, lines 82–96) — a hand-built, ordered table mapping recognizable keyword sets ("recognition, vision, cnn" → "Computer Vision") to a human-readable label. The comment explicitly says to keep this list "small, deterministic, and easy to defend in a thesis defense" — every rule maps to a domain noun a real thesis panel would recognize, rather than an opaque auto-generated label. 4. **Trend classification** (`_classify_trend`) has **two regimes**: for a small corpus (< 15 total theses) it uses simple fixed thresholds (≥ 5 theses = Saturated, 2–4 = Emerging, 1 = Underexplored); once the corpus grows past 15, it switches to **relative** scaling against the *average* cluster size ($\geq 1.5 \times$ average = Saturated, $\leq 0.5 \times$ average = Underexplored) — so the classification stays meaningful as the whole repository grows, instead of every cluster eventually looking "saturated" by the old fixed numbers.

backend/theses/services/preview_pdf.py — the DOCX/placeholder PDF renderer

Added for the "Full Thesis Document (Chapters 1–3) PDF & DOCX Preview Handler" fix. Two entry points:

- `render_docx_to_pdf` walks a DOCX's paragraphs directly via `python-docx` (a *second*, independent read of the file from `text_extractor.py`'s own DOCX extraction — this one needs to preserve **heading vs. body** structure, which the flat-text extractor throws away), detecting headings by checking each paragraph's Word style name ("Heading *" / "Title"), and lays the result out into a real, paginated PDF using `reportlab`.
- `render_placeholder_pdf` builds a metadata-only PDF (title, program/year, authors, abstract, and an excerpt of the stored `extracted_text`) for the rare case where the real file is missing from disk or fails to convert — so the previewer **never** returns a broken 404, only ever a PDF, even in a worst-case failure.

backend/theses/views.py — the repository's many counters

- `ThesisListView` / `ThesisSearchView`: Plain filtered browsing vs. SBERT-ranked semantic search — two separate endpoints because "browse everything, optionally filtered" and "rank by meaning against a query" are different enough operations to deserve their own request/response shapes rather than one endpoint silently switching behavior based on whether `?q=` was present.
- `_visible_queryset`: The role-based visibility rule used by every listing/search/detail endpoint — students see only approved theses (plus their *own* uploads regardless of status); faculty/administrators see everything.
- `ThesisDetailView`: Plain metadata lookup.
- `ThesisDownloadView`: Covered in depth in the previous round of work — streams PDFs inline, converts DOCX to PDF on the fly, and falls back to a placeholder PDF, never a 404.
- `ThesisUploadView`: The eight-step upload pipeline: validate the file → validate the metadata → SHA-256 hash it for duplicate detection → decide the initial workflow status from the uploader's role (student → `pending_review`, needs faculty sign-off; faculty/administrator → approved immediately) → save inside a transaction → **best-effort** text extraction (a failure here is logged but never blocks the upload from succeeding) → **best-effort** SBERT embedding generation (same never-block guarantee) → respond with the full detail shape. `_coerce_list_fields` (lines 459–492)

exists because `multipart/form-data` can only carry strings, not real JSON arrays — so `authors/keywords` arrive as either a JSON-encoded string or a comma-separated fallback, and this helper normalizes either shape into a real Python list before validation runs.

- `ThesisValidateTitleView / ThesisTopicTrendsView / ThesisAnalyticsView`:

Thin HTTP wrappers around the three AI services above — each does parameter validation, calls the service, and serializes the result; none of them contain any AI logic themselves (that all lives in `services/`). `ThesisAnalyticsView` is explicitly documented as "no ML, no AI inference" — pure Django ORM aggregation (Count, date-range filters, a Python Counter over the JSON keyword arrays) — deliberately kept separate from the AI-driven Trend Analysis page so the two pages' numbers can never be confused with each other.

- `ThesisExtractTitleView`: A stateless helper (nothing is saved to the

database) that reuses `ThesisTextExtractor`, then scores each line of the extracted text against a heuristic (uppercase/title-case, word count in a plausible title range, contains common thesis-title keywords like "system" or "framework," penalized if the document appears to start directly with chapter headings rather than an actual title page) to guess the most likely thesis title — returning a high/medium/low confidence label the frontend uses to warn the user to double-check a low-confidence guess.

backend/theses/urls.py / admin.py / apps.py

- `urls.py` mounts ten routes under `/api/v1/theses/` — list, public

stats, upload, search, validate-title, extract-title, topic-trends, analytics, detail, download.

- `admin.py` gives Django admin a full-featured `ThesisAdmin`: colored

status badges, an extracted-text preview, embedding-vector info, and bulk actions to approve/reject/reset-to-pending selected theses. The `approve_theses` bulk action (lines 162–208) is worth noting: after bulk-approving, it **automatically retries SBERT embedding generation** for any newly-approved thesis that doesn't already have one — closing a real lifecycle gap where a student's thesis got approved but its embedding had failed at upload time, which would otherwise leave it silently invisible to semantic search and title similarity forever.

- `apps.py` is a minimal `AppConfig` with no custom `ready()` hook.

■ `theses/management/commands/` holds three Django management commands (`seed_demo_data`, `seed_theses`, `embed_theses`) — developer tools run via `python manage.py <command>` to seed a demo dataset or backfill missing embeddings. They're operational/developer tooling rather than part of the live request-handling path, so they're mentioned here rather than walked through line-by-line.

Part 15 — Backend: the audit app (the restaurant's security diary) ■

Every "interesting" thing that happens anywhere in THESYS+ — logins, failures, password resets, access-request decisions, identity-verification pipeline runs — gets written into one shared, append-only diary. This app owns that diary's table and its one read endpoint.

backend/audit/models.py — "The diary itself"

`AuditLog` — one row per event. `actor_user` (who did it — nullable, since some events are system-initiated, like an automatic OCR-based approval) and `target_user` (who it was done **to**) are separate fields, since an administrator approving someone else's access request has a different actor than target. `ip_address/user_agent` capture **where the request came from**; `metadata` is a flexible JSON blob holding whatever event-specific details matter (a reset-token ID, a rejection reason, an OCR confidence score, ...) without needing a different table shape for every kind of event. The docstring is explicit: **application code only ever INSERTs** — nothing in the running app ever updates or deletes a row; that's

reserved for separate, deliberate retention/cleanup maintenance.

backend/audit/views.py — "The one counter where you can read the diary"

`AuditLogListView` (GET `/admin/audit-log/`) — administrator-only, paginated, and filterable by event type, actor, target, and a `created_at` date range. Every filter is validated defensively (an invalid UUID or malformed ISO 8601 datetime returns a clean 400 error rather than a raw exception) since this endpoint is a genuine investigative tool an administrator might hand-type query parameters into.

backend/audit/serializers.py / urls.py / admin.py / apps.py

- `AuditLogSerializer` — every field is read-only; there is no write path

through the API at all (the only way a row gets created is via `common.audit_logger.write()`, called from inside other apps' own service functions).

- `urls.py` mounts the single list endpoint at `/admin/audit-log/`.
 - `admin.py`'s `AuditLogAdmin` explicitly overrides `has_add_permission`, `has_change_permission`, and `has_delete_permission` to all return `False` — administrators can **browse** the diary through Django admin too, but the admin UI itself is physically incapable of adding, editing, or deleting an entry, reinforcing the append-only guarantee at every possible entry point, not just the custom API view.
 - `apps.py` is a minimal `AppConfig` — no custom `ready()` logic; this app's whole job is **storage**, not permissions.
-

Part 16 — Backend: common/ (the kitchen's shared toolbox) ■

Nothing in here is specific to logins, theses, or access requests — every app in the kitchen reaches for these same tools.

backend/common/audit_logger.py — "The pen that writes in the diary"

`write()` is the **one** function every app calls to add a row to the `AuditLog` table from Part 15. Two defensive design choices worth highlighting: it's wrapped in a broad `try/except` that **swallows any failure** and just logs it instead of raising — the comment is explicit that "a transient DB hiccup can't break the auth flow the audit entry was supposed to record" (i.e., recording **that** someone logged in should never be able to prevent them from actually logging in). And `_resolve()` (lines 57–66) accepts either an already-loaded `User` object **or** just a raw ID/UUID — so callers throughout the codebase never need to remember which shape they have on hand before writing an audit entry.

backend/common/csrf.py — "Are you sure you're calling from our own dining room?"

`require_origin_match` is the decorator seen guarding `LoginView`, `ResetPasswordView`, `SetupPasswordView`, and others throughout this document. Because THESYS+'s cookie-bearing endpoints (login, refresh, password reset, etc.) don't rely on Django's usual session-cookie-bound CSRF token, this decorator does the equivalent job a different way: it reads the browser's `Origin` header (falling back to `Referer` if `Origin` is missing) and rejects the request unless it matches one of the explicitly configured allowed origins — GET/HEAD/OPTIONS requests pass through unchecked since they can't change any state anyway.

backend/common/errors.py — "Every error looks the same shape, no matter which counter you're at"

- `make_error_response` builds the standard `{"error": {"code", "message", "details"}}` JSON body used by every hand-written error response you've seen throughout this document (`INVALID_CREDENTIALS`, `RATE_LIMITED_...`, `EMAIL_ALREADY_REGISTERED`, and so on).
- **A family of custom exceptions** (`RefreshTokenExpired`, `RefreshReuseDetected`, `AccessTokenExpiredException`, `OriginNotAllowed`, `CsrfFailure`, `InsufficientRole`) each carry their own `default_code` — this is what lets the frontend's `client.js` interceptor recognize `ACCESS_TOKEN_EXPIRED` specifically and silently refresh, rather than every 401 looking identical.
- `unified_exception_handler` — registered as Django REST Framework's global `EXCEPTION_HANDLER` setting, this is what guarantees the `{"error": {...}}` shape even for exceptions **nobody explicitly formatted** — a validation error from a serializer, DRF's own built-in `NotFound/PermissionDenied`, or a throttling exception all get funneled through here and reshaped into the same envelope, so the frontend never has to special-case "did this error come from custom code or from DRF itself."

backend/common/ratelimit.py — "No more than N tries per minute, please"

Four decorators, all built on the same `_hit_and_check` primitive (a simple counter in Django's cache backend, seeded on first hit and incremented on each subsequent one within the same time window):

- `rate_limit_per_email / rate_limit_per_ip` — general-purpose budgets keyed on whichever identifier makes sense for the endpoint (forgot-password by email; login attempts by IP).
- `rate_limit_per_email_on_failure` — a subtler one, used specifically on `LoginView`: it only counts **failed** (401) login attempts against the budget, and a **successful** login (200) resets the counter entirely. This is deliberate — if it counted every attempt equally, a legitimate user who mistypes their password twice then gets it right would still be one step closer to being locked out for no good reason; only *actual* wrong-password attempts should burn down the budget.
- `rate_limit_per_token` — keys the budget on a hash of the *token* value itself (never the plaintext) rather than IP or email — used on `reset-password/setup-password` so repeated guesses against one specific token are capped independent of who's making them.
- `_get_debug_limits` — when `DEBUG=True` (local development), every limit is relaxed to a flat 20 requests/minute regardless of what the production numbers are, so a developer manually testing a flow all afternoon doesn't lock themselves out.

backend/common/validators.py — "The stock rules every form borrows"

Three small, widely-reused validators: `is_institutional_email` (any `*.pampangastateu.edu.ph` address — used for sign-in, forgot-password, and anywhere an *existing* account's email is being checked), `is_student_number_email` (the **stricter** `<digits>@pampangastateu.edu.ph` format — used only where a document-verified student number is expected, i.e. the access-request document-upload flow), and `validate_password_strength` (12+ characters, at least one letter, at least one digit — the exact same rule every password form across the whole frontend also checks client-side, but enforced here as the real, un-bypassable authority).

backend/common/tokens/ — the three flavors of secret THESYS+ issues

- `opaque.py` — `generate_opaque_token()` makes a 256-bit random string

(used for refresh tokens, password-reset/setup tokens, and email- verification tokens alike); `sha256()` is the one-way hash every one of those token tables stores **instead of** the plaintext, so a stolen database backup alone can never be used to reconstruct a working token.

- `jwt.py` — `issue_access_token/verify_access_token` handle the

short-lived JWT access token (the "Bearer" key from Part 3/4): signed HS256, carrying the user's ID and role directly in the payload (so most requests don't need a database round-trip just to know who's asking), with a `kid` header included specifically so a future secret-key rotation has a clean way to identify which key signed an older still-valid token. `verify_access_token` distinguishes an **expired** token from an **otherwise-invalid** one via two different exception types — that distinction is exactly what lets the frontend's `client.js` tell "just refresh silently" apart from "this token is garbage, force a real re-login."

- `cookies.py` — `set_refresh_cookie/clear_refresh_cookie` centralize

every cookie attribute (`HttpOnly`, `Secure`, `SameSite=Lax`, a narrow `Path` scoping the cookie to only the auth endpoints that need it) in one place, so no individual view can accidentally set the refresh cookie with a weaker configuration than the rest of the app. `_secure_flag` (lines 16–22) deliberately turns **off** the `Secure` flag only in local development (`DJANGO_ENV=development`) — `Secure` cookies are refused by browsers over plain `http://localhost`, so without this carve-out no contributor could test login locally at all.

backend/common/email_backend.py — "Two ways to hand over the message: shout it in the kitchen, or actually mail it"

Already referenced throughout the onboarding-flow parts of this document — worth its own short recap here as the shared plumbing. Defines an `EmailBackend` protocol (`send(to, subject, template_name, context) -> bool`) with two concrete implementations selected by the `EMAIL_BACKEND_CHOICE` setting: `ConsoleEmailBackend` (renders the email template and prints it straight to the server's log — the default in local development, so a contributor can just read the verification link off their own terminal instead of needing a real mailbox) and `SMTPEmailBackend` (sends via real SMTP credentials for staging/production). If `smtp` is selected but credentials are incomplete, it **automatically falls back** to the console backend with a loud warning rather than crashing every email- sending code path — "development never breaks" is the explicit design goal in the docstring.

Part 17 — Backend: `thesys/` (the building's blueprint and utility panel) ■■

Not a feature app at all — this is Django's **project** package: the wiring that turns all the individual apps above into one running server.

backend/thesys/settings/__init__.py — "Which rulebook are we using today?"

A tiny but important indirection: whatever code (or `manage.py`, or `wsgi.py`) imports `thesys.settings`, this file quietly decides whether that actually means `thesys.settings.dev` or `thesys.settings.prod`, based on the `DJANGO_ENV` environment variable — defaulting to `development` if unset or unrecognized, "to keep local tooling forgiving."

backend/thesys/settings/base.py — "The rules every environment agrees on"

The shared settings both `dev.py` and `prod.py` build on top of. A few details worth knowing, since they explain behavior seen throughout this whole document:

- `INSTALLED_APPS` **ordering (lines 62–84)** is deliberately sequenced by

dependency direction — `accounts` first (everything else refers to its `User` model), then `audit` (only depends on `accounts`), then the three sibling apps.

- `AUTH_USER_MODEL = 'accounts.User'` — this is *the* setting that makes the whole custom, email-keyed, UUID-based `User` model from Part 10 the one Django actually uses everywhere, instead of its own built-in user model.
- `PASSWORD_HASHERS` (**lines 192–198**) lists Argon2id **first** — this is the "current best hasher" that `accounts.services.verify_password`'s `needs_rehash` logic (Part 10) compares every login attempt's stored hash against. The remaining entries (PBKDF2, bcrypt, scrypt) exist purely so that if an *old* hash from one of those algorithms is ever encountered, it can still be verified — just flagged for silent upgrade.
- `REST_FRAMEWORK[ 'EXCEPTION_HANDLER' ]` points at `common.errors.unified_exception_handler` — this single line is what makes *every* DRF error across the entire API return the same `{"error": { ... }}` shape, project-wide, without every individual view needing to remember to catch and reformat its own exceptions.
- ***JWT / refresh-cookie / rate-limit / email / identity-verification settings*** (lines 284–408) are the actual numeric knobs behind numbers quoted throughout this document — 15-minute access tokens, 24-hour/30-day refresh cookies, 75%/60% OCR confidence thresholds, and so on — all overridable via environment variables so a deployment can tune them without touching code.

backend/thesys/settings/dev.py / prod.py — "Loosen the belt at home, tighten it at the restaurant"

- `dev.py` turns `DEBUG` on, opens `ALLOWED_HOSTS` to `'*'` (so a phone on the same Wi-Fi, or a tunneling tool like ngrok, can reach a locally-run backend without extra config), and routes the `emails/audit/ access_requests` loggers to the console — this is precisely what makes the `ConsoleEmailBackend` banners (with the clickable verification link) show up in a developer's terminal.
- `prod.py` does the opposite: `DEBUG` forced off, `ALLOWED_HOSTS` must be explicitly supplied (an empty list — the safe default — makes Django reject every request with a 400 until an operator configures real hosts), and a handful of cookie/header security flags are tightened. The comment is refreshingly honest about what's **not** yet enabled: HSTS and forced HTTPS redirection are deliberately deferred until a real TLS-terminating reverse proxy is in front of the app — turning them on prematurely would make even a valid plain-HTTP smoke test fail.

backend/thesys/urls.py — "The building directory in the lobby"

The root URL map that ties every app's own `urls.py` together: cookie-bearing public auth endpoints (login/logout/refresh, forgot/reset/setup- password, public request-access) all under `/api/v1/auth/`; the thesis repository under `/api/v1/theses/`; administrator-only endpoints (access- request approval/denial, the audit log) under `/api/v1/admin/`. The comment explicitly notes `accounts` has **no** URL entry here at all — as established in Part 10, that app is purely a model-holder.

backend/thesys/health.py — "Is the lobby door even open?"

The simplest possible view: `GET /api/v1/health` returns a static `{"status": "ok"}`, with **no** authentication, rate limiting, or CSRF check — deliberately, since its entire purpose is letting an ops/monitoring tool (or a deploy script) confirm "the server process is up and the URL routing works" without needing any credentials at all.

backend/thesys/wsgi.py / asgi.py — the two "ways in" for a real server

Both are nearly-identical few-line files that Django's project template generates automatically — `wsgi.py` is what a traditional synchronous server (Gunicorn, uWSGI) talks to; `asgi.py` is the equivalent entry point for an async-capable server. THESYS+ doesn't currently use any async-specific Django features, so `asgi.py` exists mainly for forward compatibility/deployment flexibility rather than because anything in the app actually requires it.

backend/manage.py — "The front-desk command console"

The standard Django CLI entry point every command in this whole document runs through — `python manage.py runserver`, `python manage.py check`, `python manage.py migrate`, `pytest` (via `pytest-django`, which reads `DJANGO_SETTINGS_MODULE` the same way), and every management command mentioned in Part 14. Its only real logic is defaulting `DJANGO_SETTINGS_MODULE` to `thesys.settings` (which, as covered above, then quietly resolves to `dev` or `prod`) and giving a friendly error message if Django itself isn't installed/importable — the classic "did you forget to activate your virtual environment?" hint.

Closing note — scope of this document

This document now walks through every actively-used file across both `frontend/src/` and `backend/` (excluding auto-generated Django migrations, the automated test suites, and a handful of standalone developer diagnostic scripts in `backend/` — none of those are part of the live request-handling path, so they were intentionally left out of a line-by-line walkthrough rather than accidentally forgotten). Several concrete audit findings turned up along the way. Two stale copy references to the eliminated "second email" (`RequestAccessPage.jsx` and `LegalModal.jsx`'s Help content) were corrected as part of this audit. Five small orphaned/unused components (`ForgotPasswordForm`, `ResetPasswordForm`, `SsoButton`, `MainLayout/ThemeToggle`, `ui/Logo`) were left in place — they're harmless dead code, not bugs — but flagged in the relevant Part above as candidates for a future cleanup pass.

Document complete.